Lógica y Representación 1

Ordenamiento y búsqueda en arreglos

Ingeniería de Sistemas

Universidad de Antioquia

Método de Polya - Resumen





PROBLEMA

ALGORITMO



1. Entender el problema
2. Diseñar un plan
3. Implementar el plan
4. Revisar el plan

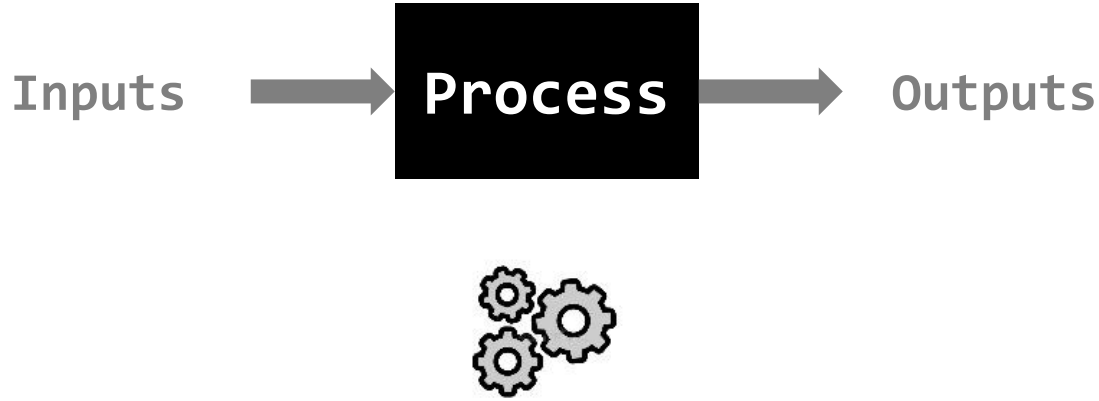
Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**

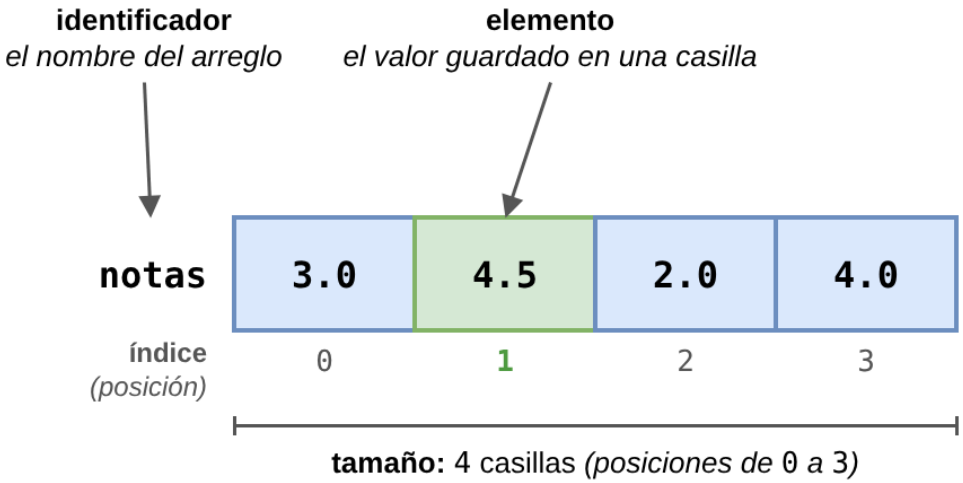






Repaso: la lista usada como arreglo

- Un arreglo guarda varios datos **del mismo tipo** bajo un solo nombre.
- Cada dato se identifica por su **posición**, de 0 a N − 1.
- Se recorre con un ciclo, usando el índice como variable.



notas[1] → **4.5** *el elemento de la posición 1*

primera posición: `0` *última posición:* `N - 1 = 3`

Pseudocódigo

```
notas = [3.0, 4.5, 2.0, 4.0]
N = 4
Para (i = 0, N - 1, 1)
  Escribir(i, notas[i])
Fin_Para
```

Python

```
notas = [3.0, 4.5, 2.0, 4.0]
N = 4
for i in range(N):
    print(f"Posición {i}: {notas[i]}")
```

Link simulación

Repaso: funciones con arreglos

Una función puede recibir un arreglo (en Python, una lista) y:

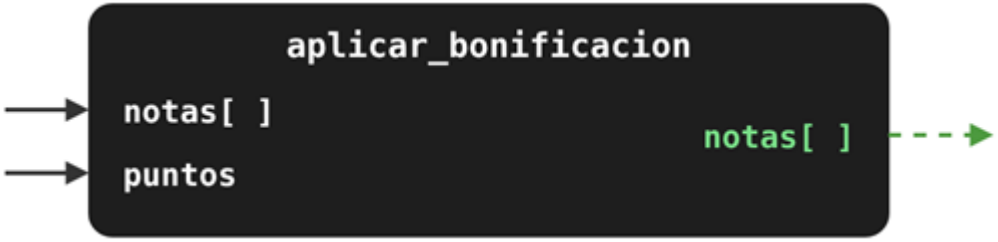
1. Retornar un valor (simulación).



2. Retornar una lista nueva (simulación).



3. Modificar la lista que recibe (sin `return`) (simulación).



Los métodos de ordenamiento de esta clase son del "caso 3": modifican la lista que reciben.

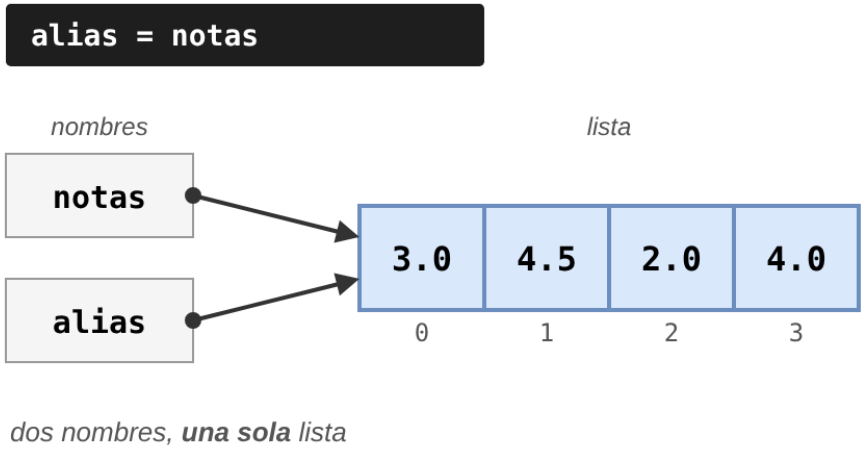
Repaso: Alias y copia de una lista

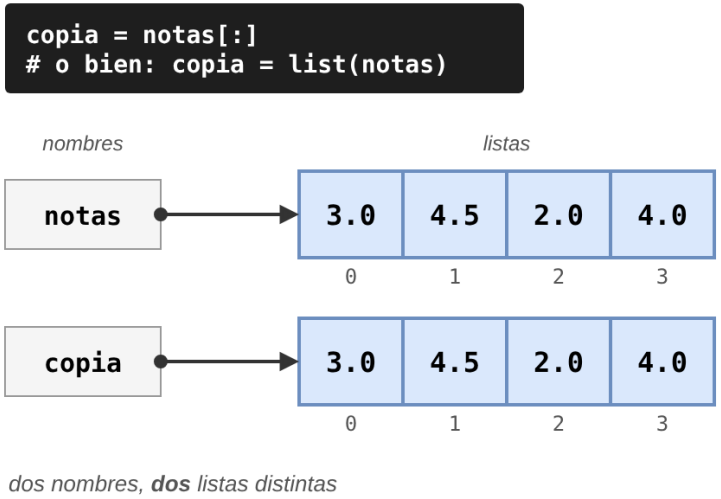
- **`B = A` no copia**: los dos nombres se refieren a la misma lista (alias).
- **`B = list(A)` o `B = A[:]` sí copia**: crea una lista nueva.
- Ordenar en el lugar modifica la lista original. Si se necesita conservar el original, primero se hace una copia.

```
notas = [3.0, 4.5, 2.0, 4.0]
alias = notas           # no copia: es la misma lista
copia = list(notas)     # sí copia: es una lista nueva

notas[0] = 5.0
print(notas)    # [5.0, 4.5, 2.0, 4.0]
print(alias)    # [5.0, 4.5, 2.0, 4.0]
print(copia)    # [3.0, 4.5, 2.0, 4.0]
```

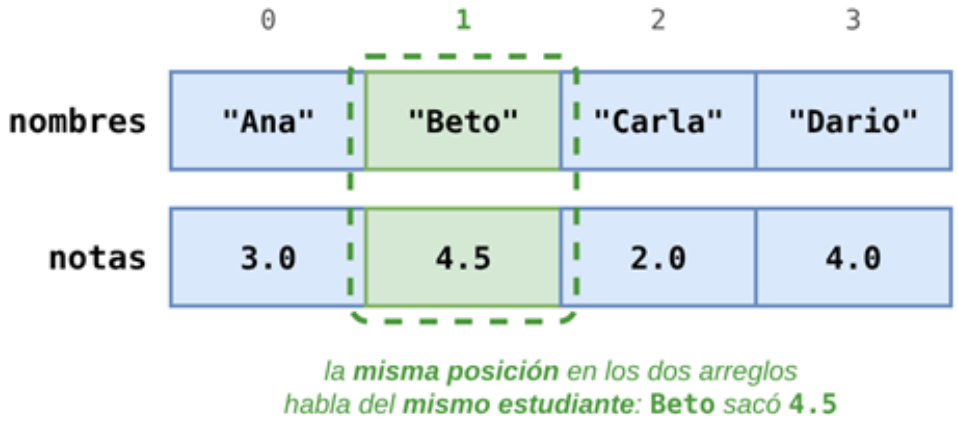
Link simulación





El problema pendiente: arreglos paralelos

- Al final de la clase 11 quedó un problema: los **`nombres`** y las **`notas`** de un curso se guardan en **dos arreglos paralelos**. La misma posición corresponde al mismo estudiante.




```
nombres = ["Ana", "Beto", "Carla", "Dario"]
notas = [3.0, 4.5, 2.0, 4.0]
N = 4
for i in range(N):
    print(f"{nombres[i]}: {notas[i]}")
```

Link simulación

- **Preguntas que todavía no se pueden responder bien**:
  1. ¿Cómo se muestra el listado de mayor a menor nota?
  2. ¿Cuál es la nota de **Carla**?
  3. Si se ordena **`notas`**, ¿cómo se mantiene cada nombre unido a su nota?

- La primera y la tercera piden **ordenar**; la segunda pide **buscar**.

¿Dónde se ordena?

- Ordenar es reorganizar los datos según un **criterio**, llamado **clave**, en orden **ascendente** (de menor a mayor) o **descendente** (de mayor a menor).
- **Ejemplos cotidianos**:

Aplicación	**Qué se ordena**	**Clave**	**Orden**
Música	Canciones	Título, artista o fecha	Ascendente
Correo	Mensajes	Fecha	Descendente: el más reciente primero
Tienda en línea	Productos	Precio	Ascendente o descendente
Explorador de archivos	Archivos	Nombre, fecha o tamaño	Ascendente o descendente
Este curso	**Estudiantes**	**Nota**	**Descendente**

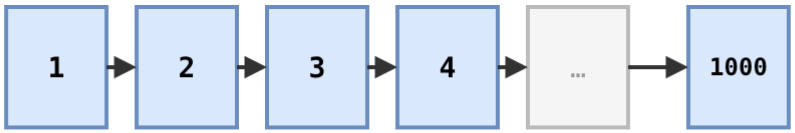
- **Ejemplo**: Ordenar `[2, 8, 9, 6]`
  - **Ordenamiento en orden ascendente**: `[2, 6, 8, 9]`
  - **Ordenamiento en orden descendente**: `[9, 8, 6, 2]`

¿Buscar en desorden o en orden?

- Una guía telefónica de 1000 páginas, ordenada por apellido. Se busca a “Pérez”.
- **Estrategias de búsqueda**:
  1. **Búsqueda lineal (página por página)**
  2. **Búsqueda binaria (abrir por la mitad)**

Guía telefónica de **1000** páginas, ordenada por apellido. Se busca a **"Pérez"**.

1. Página por página

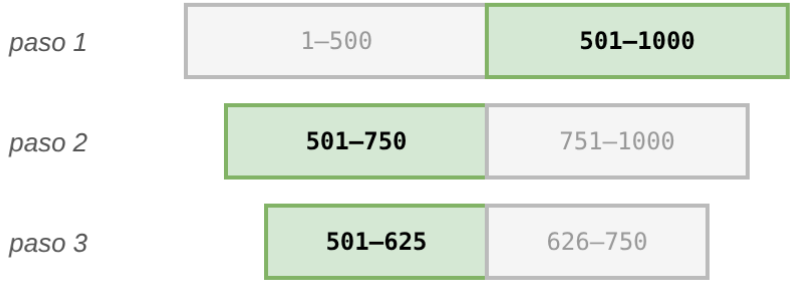


se revisa una página tras otra

- hasta **1000** pasos
- sirve aunque la guía esté **en desorden**

búsqueda lineal

2. Abrir por la mitad



`1000 → 500 → 250 → 125 → … → 1 página`

verde: la mitad que se conserva · gris: la que se descarta

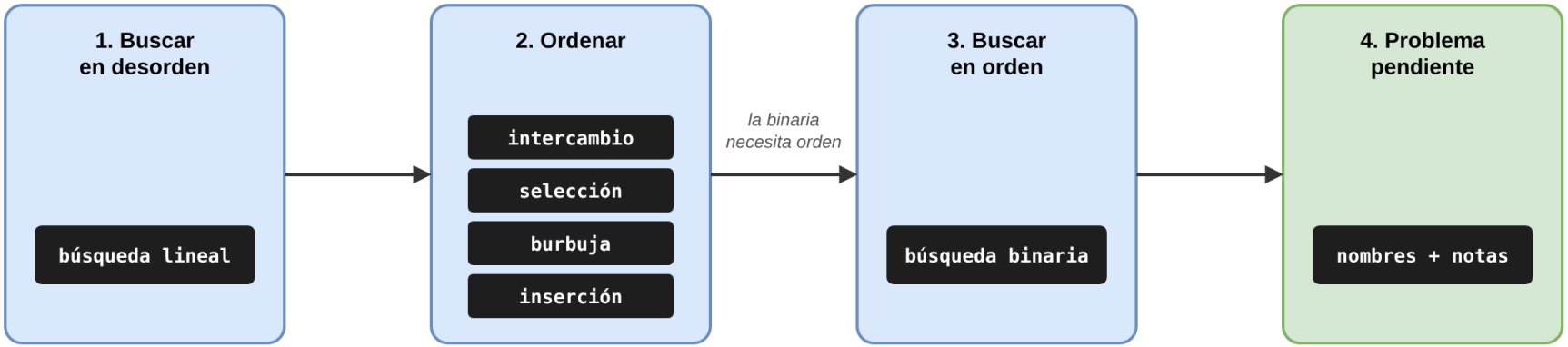
- unos **10** pasos
- solo si está **ordenada**

búsqueda binaria

Idea tomada de CS50x, Lecture 3 (link)

Plan de la clase

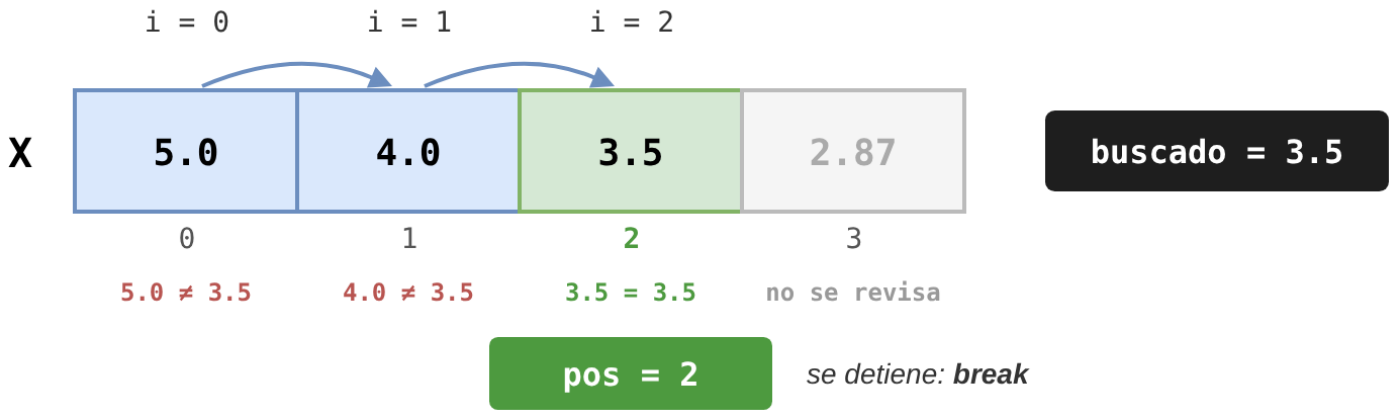
1. **Buscar en desorden**: búsqueda lineal.
2. **Ordenar**: intercambio y tres métodos (selección, burbuja e inserción).
3. **Buscar en orden**: búsqueda binaria.
4. **Resolver el problema pendiente**: ordenar y buscar en los arreglos paralelos **`nombres`** y **`notas`**.



Todo se implementa con funciones: caja negra, Supone/Garantiza y pruebas con **`assert`**

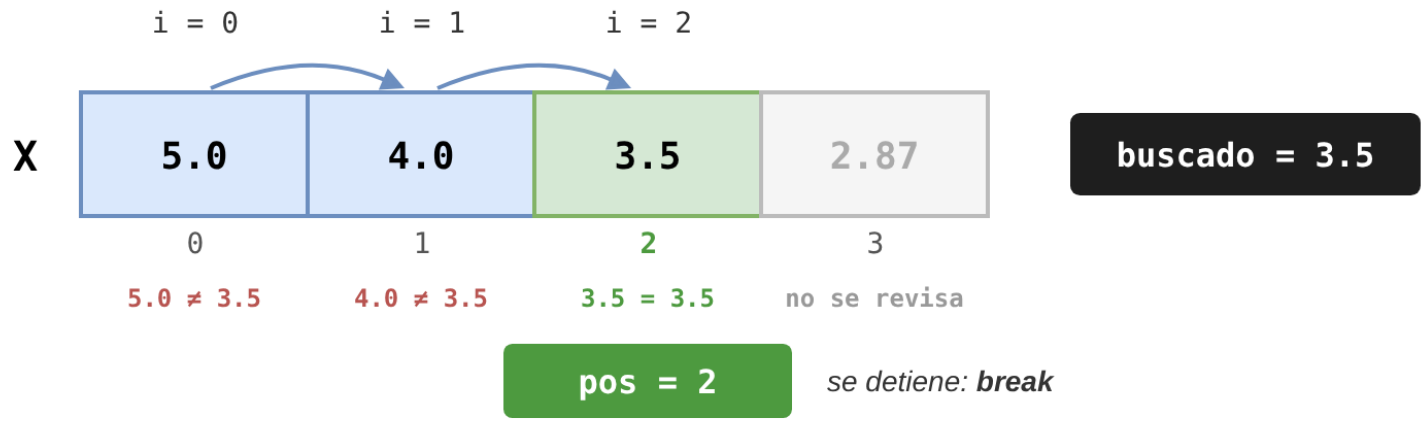
Búsqueda lineal

- Se recorre el arreglo desde la posición **`0`** y se compara cada elemento con el valor buscado.
- Al encontrarlo, se **guarda su posición** y se detiene el recorrido (**`break`**).
- Si se recorre todo el arreglo sin encontrarlo, la posición queda en **`-1`**, que significa "**no está**". El **`-1`** sirve porque ninguna posición válida es negativa.
- Funciona aunque el arreglo **no esté ordenado**.



*Si el valor no estuviera, se revisarían todas las casillas y **pos** quedaría en **-1**.*

Búsqueda lineal



*Si el valor no estuviera, se revisarían todas las casillas y **pos** quedaría en **−1**.*

Pseudocódigo

```
X = [5.0, 4.0, 3.5, 2.87]
N = 4
buscado = 3.5
pos = -1
Para (i = 0, N - 1, 1) Haga
    Si (X[i] == buscado) Entonces
        pos = i
        Romper
    Fin_Si
Fin_Para
Si (pos == -1) Entonces
    Escribir("El valor NO se encontró")
Sino
    Escribir("El valor encontrado en ", pos)
Fin_Si
```

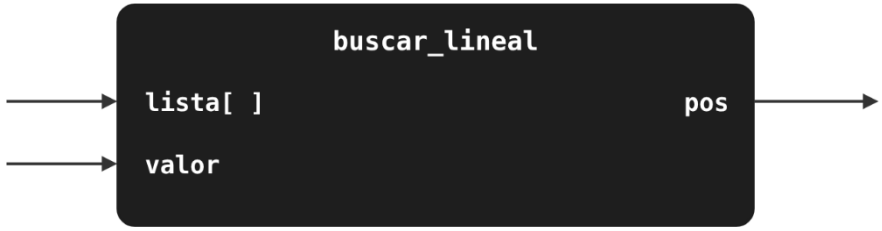
Python

```
X = [5.0, 4.0, 3.5, 2.87]
N = 4
buscado = 3.5
pos = -1
for i in range(N):
    if X[i] == buscado:
        pos = i
        break
if pos == -1:
    print("El valor NO se encontró")
else:
    print(f"El valor encontrado en {pos}")
```

Link simulación

Búsqueda lineal como función

- El algoritmo se empaqueta en una función que recibe una **lista** y un **valor** y retorna una **posición**.
- Dentro de la función se usa **`len(lista)`**, porque la función no recibe **`N`**
- La especificación dice qué supone la función, qué garantiza y que no modifica la lista.



ejemplo: `buscar_lineal([5.0, 4.0, 3.5, 2.87], 3.5)` *devuelve* **2**

```
def buscar_lineal(lista, valor):
    """
    Busca un valor en una lista, recorriéndola desde la posición 0.

    Supone:
        lista es una lista (no necesita estar ordenada).
    Garantiza:
        Retorna la posición de la primera aparición de valor en lista,
        o -1 si valor no está. No modifica lista.
    """
    pos = -1
    for i in range(len(lista)):
        if lista[i] == valor:
            pos = i
            break
    return pos
```

Link simulación

Pruebas de `buscar_lineal`

- Las pruebas se planean antes con casos típicos, límite y especiales, y se escriben con **`assert`**.

Caso	Tipo	Resultado esperado
`3.5` en `X`	Típico: está en el medio	`2`
`5.0` en `X`	Límite: es el primero	`0`
`2.87` en `X`	Límite: es el último	`3`
`7.0` en `X`	Especial: no está	`-1`
`1` en `[1, 2, 1]`	Especial: está repetido	`0` (la primera aparición)
`3.5` en `[]`	Especial: lista vacía	`-1`

```
X = [5.0, 4.0, 3.5, 2.87]
assert buscar_lineal(X, 3.5) == 2       # típico: está en el medio
assert buscar_lineal(X, 5.0) == 0       # límite: el primero
assert buscar_lineal(X, 2.87) == 3      # límite: el último
assert buscar_lineal(X, 7.0) == -1      # especial: no está
assert buscar_lineal([1, 2, 1], 1) == 0 # especial: repetido -> la primera
assert buscar_lineal([], 3.5) == -1     # especial: lista vacía
print("Todas las pruebas pasaron")
```

Link simulación

¿Cuál es la nota de Carla?

- Se responde la **pregunta 2** del problema pendiente.
- Se busca el **nombre** en **nombres** con buscar_lineal, y la posición encontrada se usa en notas.
- Si el nombre no está, **pos** vale **–1** y no se debe usar **notas[pos]**. Con **–1**, Python devolvería la última nota sin dar error (Python soporta índices negativos).



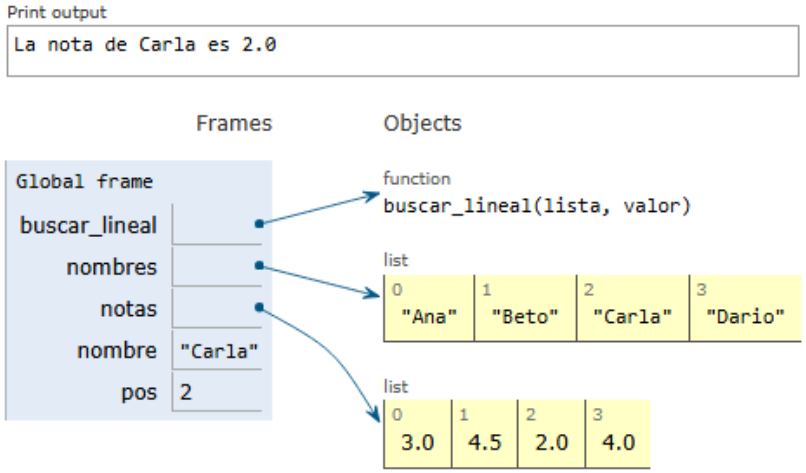
Se busca en un arreglo y se usa la posición encontrada en el otro.

Python

```
nombres = ["Ana", "Beto", "Carla", "Dario"]
notas = [3.0, 4.5, 2.0, 4.0]

nombre = "Carla"
pos = buscar_lineal(nombres, nombre)
if pos == -1:
    print(f"{nombre} no está en el curso")
else:
    print(f"La nota de {nombre} es {notas[pos]}")
```

Link simulación



Búsqueda lineal: ¿cuántas comparaciones hace?

- Con un arreglo de **N** elementos:
  - **Mejor caso**: el valor está en la primera posición. Se hace **1** comparación.
  - **Peor caso**: el valor está en la última posición o no está. Se hacen **N** comparaciones.
- **Si N crece, el peor caso crece igual**: el doble de datos son el doble de comparaciones.

Tamaño N	Mejor caso	Peor caso
4	1	4
1 000	1	1 000
1 000 000	1	1 000 000

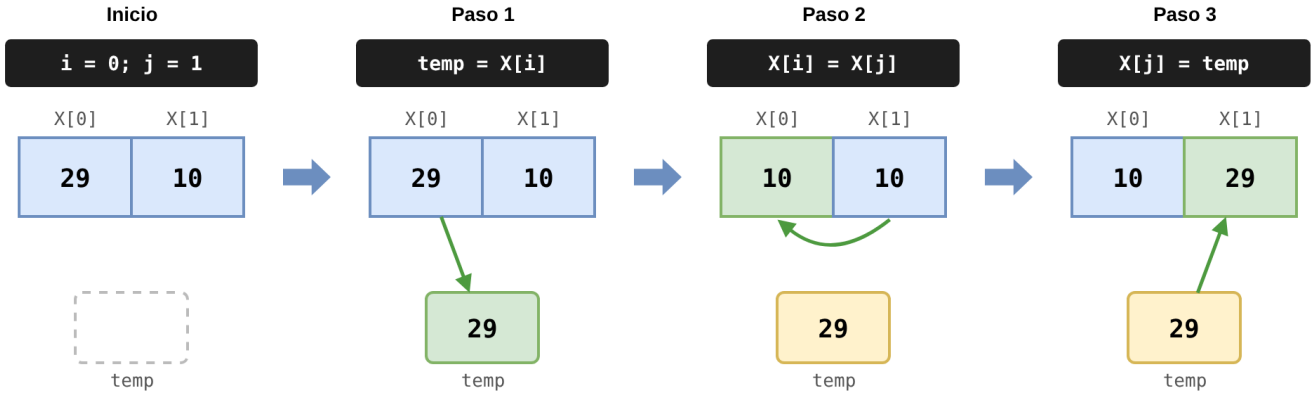
```
def contar_comparaciones_lineal(lista, valor):
    comparaciones = 0
    for i in range(len(lista)):
        comparaciones = comparaciones + 1
        if lista[i] == valor:
            break
    return comparaciones

X = [5.0, 4.0, 3.5, 2.87]
print(contar_comparaciones_lineal(X, 5.0))    # el primero
print(contar_comparaciones_lineal(X, 3.5))    # en el medio
print(contar_comparaciones_lineal(X, 2.87))   # el último
print(contar_comparaciones_lineal(X, 7.0))    # no está
```

Link simulación

Intercambiar dos posiciones de un arreglo

- Los tres métodos de ordenamiento se basan en intercambiar dos elementos del arreglo.
- Si se escribe **`X[i] = X[j]`** y luego **`X[j] = X[i]`**, el valor de **`X[i]`** se pierde (paso 2 del diagrama: 10 | 10).
- Se usa una variable temporal (**`temp`**) que guarda el valor de **`X[i]`** antes de sobrescribirlo.



Resultado: X = [10, 29, 14, 37, 13] (el 29 se guardó en temp antes de sobrescribirlo)

Pseudocódigo

```
X = [29, 10, 14, 37, 13]
i = 0
j = 1
temp = X[i]
X[i] = X[j]
X[j] = temp
Escribir(X)
```

Python

```
X = [29, 10, 14, 37, 13]
i = 0
j = 1
temp = X[i]
X[i] = X[j]
X[j] = temp
print(X)
```

Link simulación

En Python, intercambio en una línea

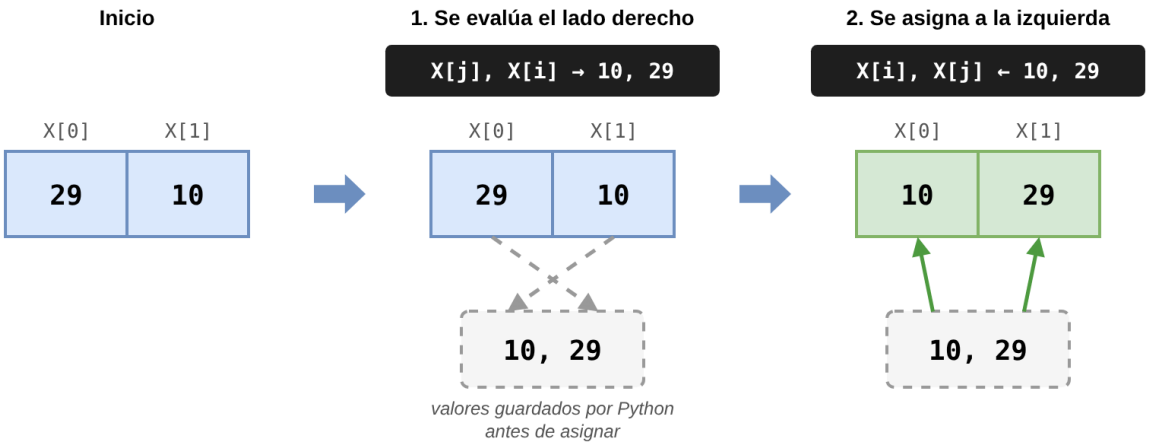
- Python permite intercambiar en una sola línea: **`X[i], X[j] = X[j], X[i]`**.
- Python **evalúa primero todo el lado derecho** y después asigna
- Otros lenguajes también lo permiten (Go, Ruby, JavaScript); C, Java y el pseudocódigo no. Por eso los algoritmos de esta clase se escriben con **`temp`**.

Uso de variable temporal

```
temp = X[i]
X[i] = X[j]
X[j] = temp
```

Uso asignación múltiple (Python)

```
X[i], X[j] = X[j], X[i]
```



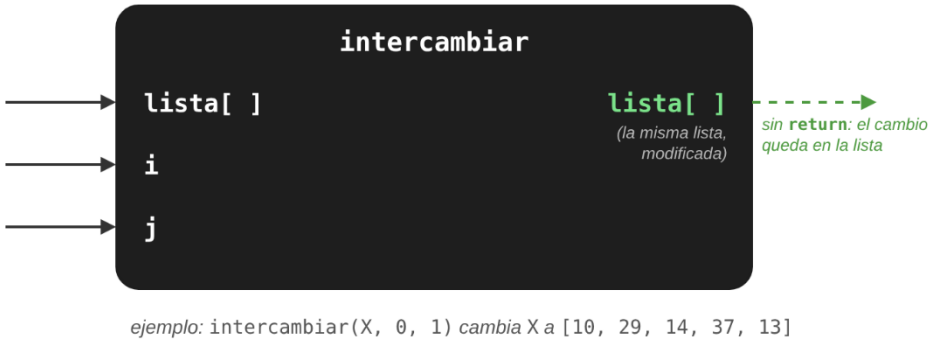

```
X = [29, 10, 14, 37, 13]
i = 0
j = 1
X[i], X[j] = X[j], X[i]
print(X)
```

Link simulación

X[i], X[j] = X[j], X[i] → X = [10, 29, 14, 37, 13] (Python usa un temporal por dentro)

El intercambio como función

- El intercambio se empaqueta en una función que recibe la lista y las dos posiciones.
- **Es del caso 3**: modifica la lista que recibe y no tiene **`return`**. Su Garantiza dice MODIFICA.
- Los tres métodos de ordenamiento la van a reutilizar.




```
def intercambiar(lista, i, j):
    """
    Intercambia los elementos de las posiciones i y j de una lista.

    Supone:
        i y j son posiciones válidas de lista (0 a len(lista) - 1).
    Garantiza:
        MODIFICA lista: el valor que estaba en i queda en j, y el que
        estaba en j queda en i. No retorna nada.
    """
    temp = lista[i]
    lista[i] = lista[j]
    lista[j] = temp
```

Link simulación

Pruebas de `intercambiar`

- Para las pruebas, se llama a la función y después se revisa la lista.
- Antes de cada prueba se vuelve a crear X, porque la función modifica la lista

Caso	Tipo	Resultado esperado
`intercambiar(X, 0, 1)`	**Típico**: vecinos	`[10, 29, 14, 37, 13]`
`intercambiar(X, 0, 4)`	**Límite**: primero y último	`[13, 10, 14, 37, 29]`
`intercambiar(X, 2, 2)`	**Especial**: la misma posición	`[29, 10, 14, 37, 13]`, sin cambios

- **Código** (**Salida**: **`Todas las pruebas pasaron`**):

```
X = [29, 10, 14, 37, 13]
intercambiar(X, 0, 1)
assert X == [10, 29, 14, 37, 13]     # típico: vecinos

X = [29, 10, 14, 37, 13]
intercambiar(X, 0, 4)
assert X == [13, 10, 14, 37, 29]     # límite: primero y último

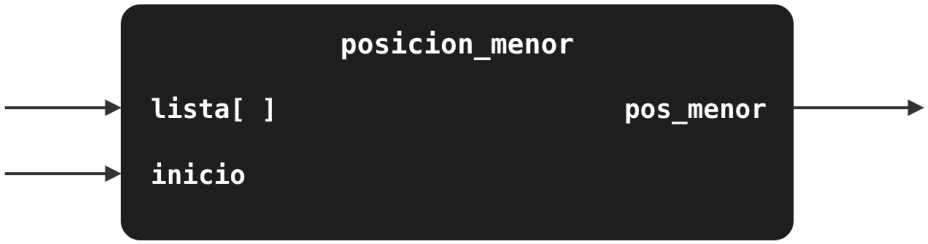
X = [29, 10, 14, 37, 13]
intercambiar(X, 2, 2)
assert X == [29, 10, 14, 37, 13]     # especial: la misma posición

print("Todas las pruebas pasaron")
```

Link simulación

La posición del menor desde una posición dada

- **La búsqueda empieza en inicio**: se inicializa **`pos_menor = inicio`** y se compara con los elementos siguientes.
- **Es del caso 1**: retorna una posición y no modifica la lista.
- El **ordenamiento por selección** la usa en cada pasada.



ejemplo: `posicion_menor([29, 10, 14, 37, 13], 2)` *devuelve* **4**

```
def posicion_menor(lista, inicio):
    """
    Busca el menor elemento desde la posición inicio hasta el final.

    Supone:
        0 <= inicio < len(lista).
    Garantiza:
        Retorna la posición del menor valor entre lista[inicio] y el
        último elemento (si se repite, la primera). No modifica lista.
    """
    pos_menor = inicio
    for j in range(inicio + 1, len(lista)):
        if lista[j] < lista[pos_menor]:
            pos_menor = j
    return pos_menor
```

Link simulación

Pruebas de `posicion_menor`

- Casos típicos, límite y especiales, escritos con **`assert`**.
- Como es del **caso 1**, también se verifica que la lista no cambió después de usarla.

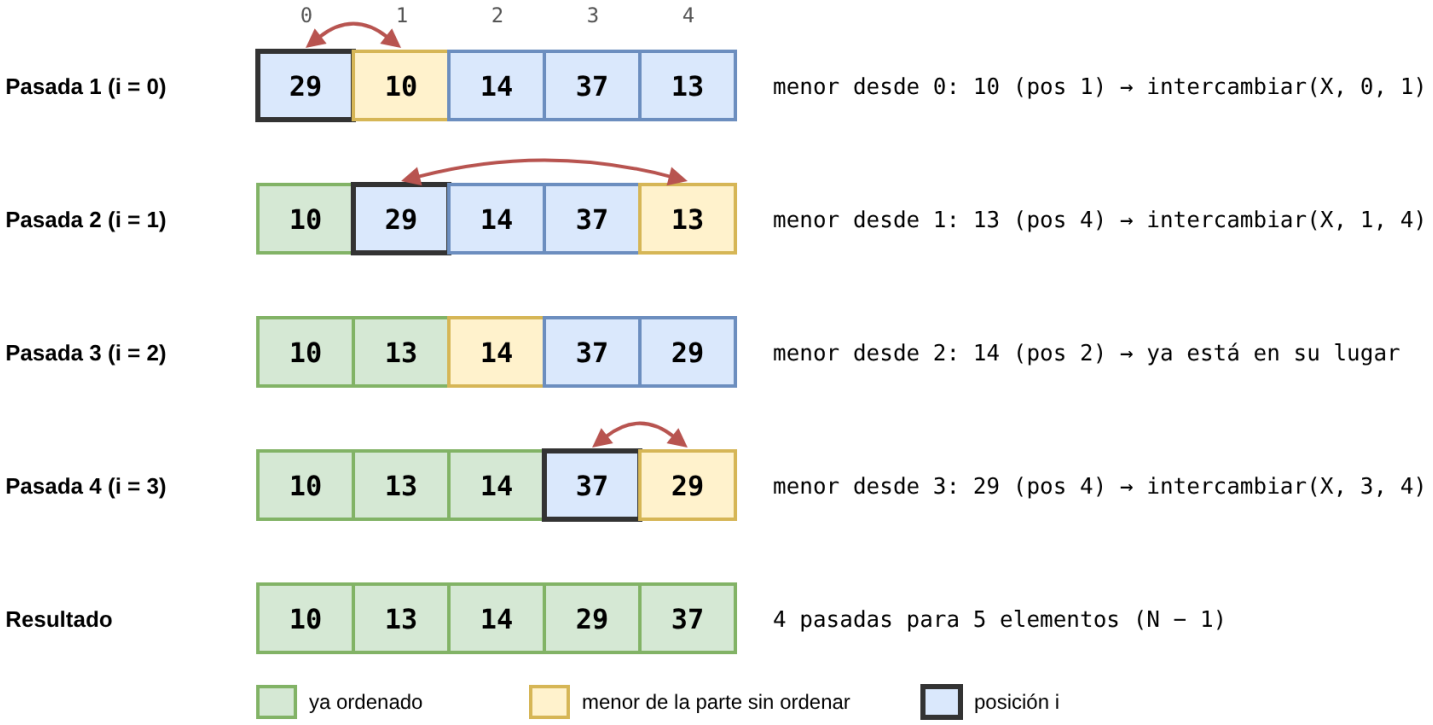
Caso	Tipo	Resultado esperado
`posicion_menor(X, 0)`	**Típico**: toda la lista	`1`
`posicion_menor(X, 2)`	**Típico**: desde el medio	`4`
`posicion_menor(X, 4)`	**Límite**: desde el último	`4` (él mismo)
`posicion_menor([5, 3, 3], 0)`	**Especial**: menor repetido	`1` (la primera aparición)
`posicion_menor([7], 0)`	**Especial**: un solo elemento	`0`

```
X = [29, 10, 14, 37, 13]
assert posicion_menor(X, 0) == 1           # típico: toda la lista
assert posicion_menor(X, 2) == 4           # típico: desde el medio
assert posicion_menor(X, 4) == 4           # límite: desde el último
assert posicion_menor([5, 3, 3], 0) == 1   # especial: menor repetido
assert posicion_menor([7], 0) == 0         # especial: un solo elemento
assert X == [29, 10, 14, 37, 13]           # no modifica la lista
print("Todas las pruebas pasaron")
```

Link simulación

Ordenamiento por selección

- En cada pasada se **busca el menor** de la parte sin ordenar y se **intercambia** con la primera posición de esa parte.
- La parte ordenada crece de izquierda a derecha, un elemento por pasada.
- Con N elementos se hacen **N – 1 pasadas**: cuando quedan N − 1 en su lugar, el último también lo está.



Analogía: de un mazo de cartas en desorden se escoge la carta más baja y se pone a la izquierda; se repite con las que quedan.

Ordenamiento por selección: el algoritmo

- **Ciclo externo (`i`)**: recorre las posiciones que se van ordenando, de **`0`** a **`N - 2`** (penúltimo elemento).
- **Ciclo interno (`j`)**: busca la posición del menor desde **`i + 1`** hasta el final.
- **Al terminar el ciclo interno se intercambia el menor con la posición `i`**: un solo **intercambio por pasada**.

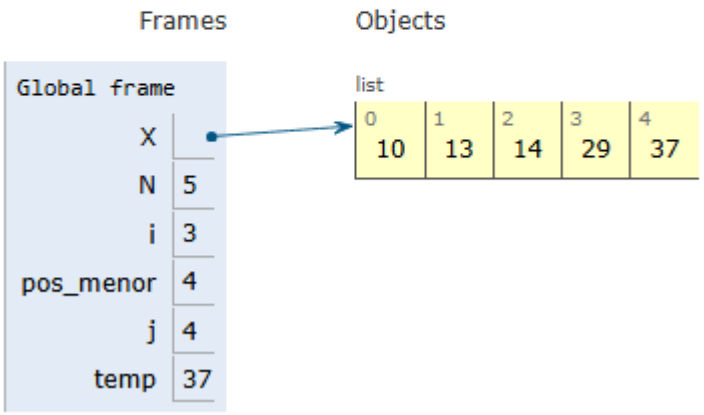
Código:

```
X = [29, 10, 14, 37, 13]
N = 5
print("inicio:", X)
for i in range(N - 1):
    # buscar la posición del menor desde i
    pos_menor = i
    for j in range(i + 1, N):
        if X[j] < X[pos_menor]:
            pos_menor = j
    # intercambiar X[i] con el menor
    temp = X[i]
    X[i] = X[pos_menor]
    X[pos_menor] = temp
    print("i =", i, "| pos_menor =", pos_menor, "|", X)
print("final:", X)
```

Link simulación

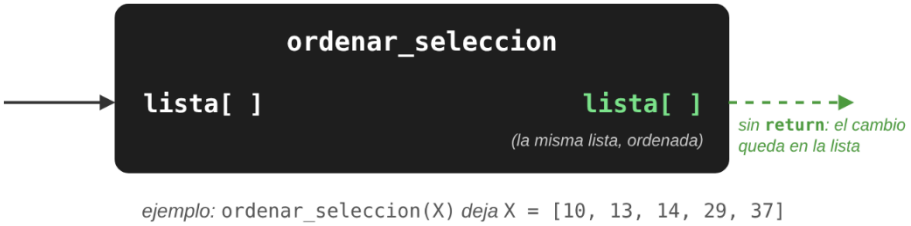
Print output

```
inicio: [29, 10, 14, 37, 13]
i = 0 | pos_menor = 1 | [10, 29, 14, 37, 13]
i = 1 | pos_menor = 4 | [10, 13, 14, 37, 29]
i = 2 | pos_menor = 2 | [10, 13, 14, 37, 29]
i = 3 | pos_menor = 4 | [10, 13, 14, 29, 37]
final: [10, 13, 14, 29, 37]
```



Selección como función

- **Cada comentario de la diapositiva anterior se convierte en una llamada a función**: el ciclo `j` pasa a ser **`posicion_menor(lista, i)`** y las tres líneas del intercambio pasan a ser **`intercambiar(lista, i, pos_menor)`**.
- **El nombre de la función reemplaza al comentario**: ya dice qué hace.
- **Caso 3**: MODIFICA la lista y no retorna nada. La lista original queda ordenada.




```
def ordenar_seleccion(lista):
    """
    Ordena una lista de menor a mayor con el método de selección.

    Supone:
        Los elementos de lista se pueden comparar con <.
    Garantiza:
        MODIFICA lista: queda ordenada de menor a mayor. No retorna nada.
    """
    for i in range(len(lista) - 1):
        pos_menor = posicion_menor(lista, i)
        intercambiar(lista, i, pos_menor)
```

Link simulación

Pruebas de `ordenar_seleccion`

- **Como la función no retorna nada, la prueba se hace en dos pasos**: primero se llama a la función y después se revisa la lista con **`assert`**.
- **Casos**: el ejemplo de la clase, una lista ya ordenada, una al revés, una con repetidos y una de un solo elemento.
- **Error frecuente**: **`assert ordenar_seleccion(X) == [...]`** falla siempre, porque la función retorna **`None`**.

```
# Ejemplo de la clase
X = [29, 10, 14, 37, 13]
ordenar_seleccion(X)
assert X == [10, 13, 14, 29, 37]

...

# Un solo elemento
X = [7]
ordenar_seleccion(X)
assert X == [7]

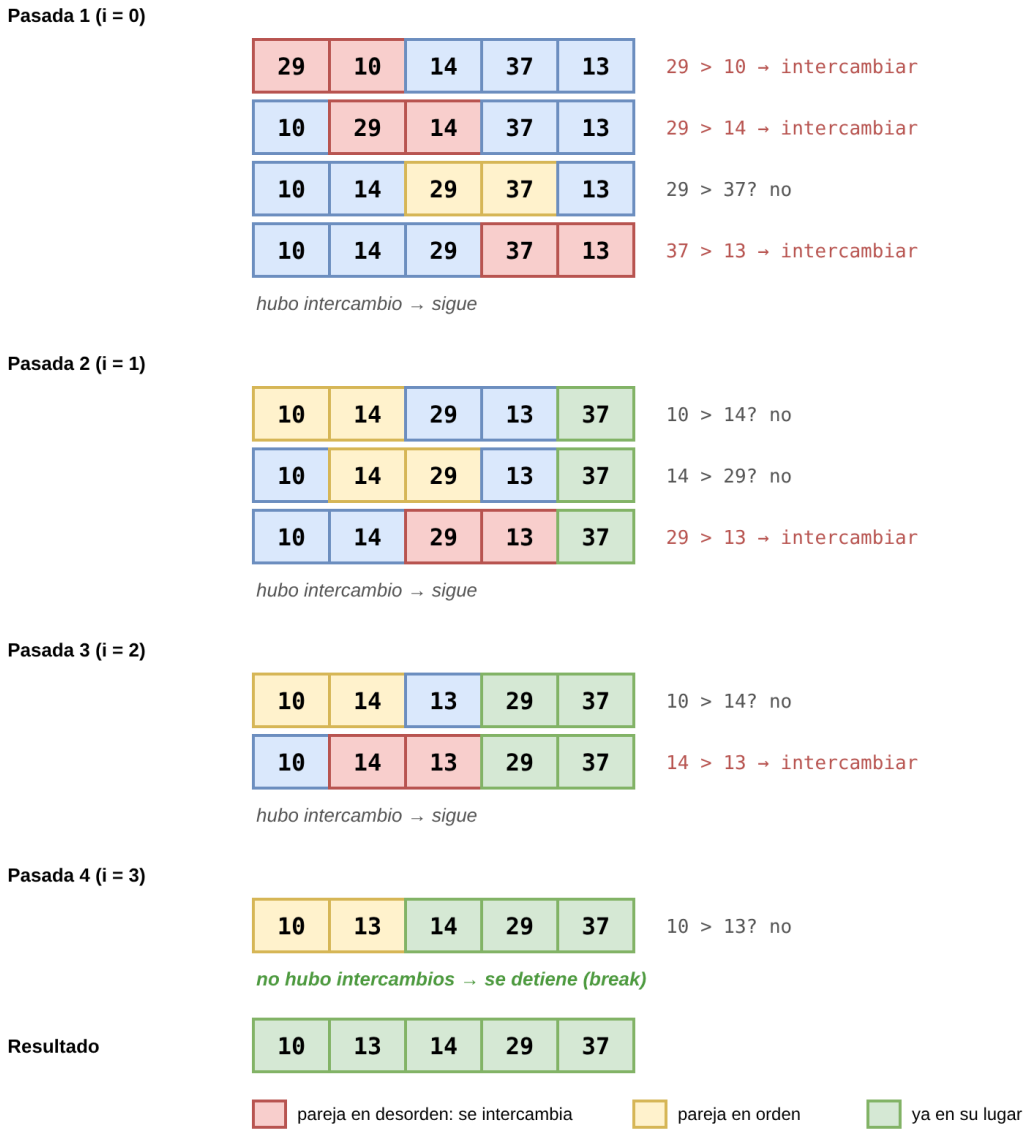
# Así NO (descomentar): la función retorna None
# X = [29, 10, 14, 37, 13]
# assert ordenar_seleccion(X) == [10, 13, 14, 29, 37]

print("Pruebas de ordenar_seleccion: todas pasaron")
```

Link simulación

Ordenamiento por burbuja

- Se recorre la lista comparando **vecinos**: si están en desorden, se intercambian.
- En cada pasada, el **mayor "sube" hasta el final**, como una burbuja. Por eso cada pasada revisa un elemento menos.
- Si en una pasada **no hubo intercambios**, la lista ya está ordenada y se detiene.



Ordenamiento por burbuja: el algoritmo

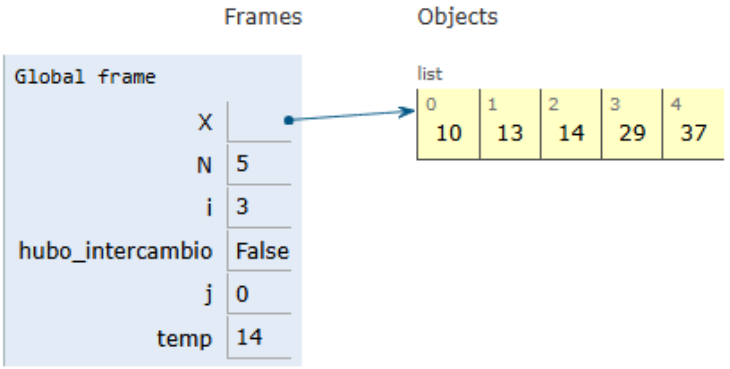
- **Ciclo interno (`j`)**: compara vecinos (**`X[j]`** y **`X[j + 1]`**) y los intercambia si están en desorden. En cada pasada, el mayor **"sube"** hasta el final.
- **Ciclo externo (`i`)**: cuenta las pasadas. En cada una se revisa un elemento menos (**`N - 1 - i`**), porque el final ya quedó ordenado.
- **`hubo_intercambio`**: si una pasada completa no intercambia nada, la lista ya está ordenada y se sale con **`break`**.

Código:

```
X = [29, 10, 14, 37, 13]
N = 5
print("inicio:", X)
for i in range(N - 1):
    hubo_intercambio = False
    # comparar vecinos e intercambiar si están en desorden
    for j in range(N - 1 - i):
        if X[j] > X[j + 1]:
            temp = X[j]
            X[j] = X[j + 1]
            X[j + 1] = temp
            hubo_intercambio = True
    print("i =", i, "| hubo_intercambio =",
          hubo_intercambio, "|", X)
    # si no hubo intercambios, ya está ordenada
    if not hubo_intercambio:
        break
print("final:", X)
```

Print output

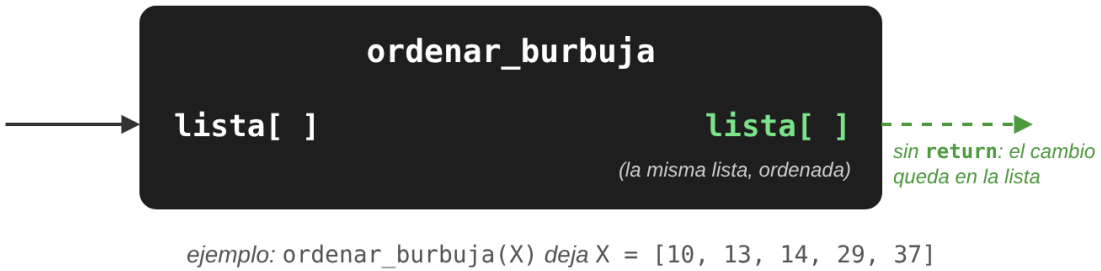
```
inicio: [29, 10, 14, 37, 13]
i = 0 | hubo_intercambio = True | [10, 14, 29, 13, 37]
i = 1 | hubo_intercambio = True | [10, 14, 13, 29, 37]
i = 2 | hubo_intercambio = True | [10, 13, 14, 29, 37]
i = 3 | hubo_intercambio = False | [10, 13, 14, 29, 37]
final: [10, 13, 14, 29, 37]
```



[Link simulación]

Burbuja como función

- **Las tres líneas del `temp` se reemplazan por `intercambiar(lista, j, j + 1)`, la misma función de la selección**: se escribió una vez y se usa en los dos métodos.
- Dentro de la función, **`N`** se reemplaza por **`n = len(lista)`**. Se guarda en una variable porque se usa dos veces.
- **Caso 3**: MODIFICA la lista y no retorna nada.



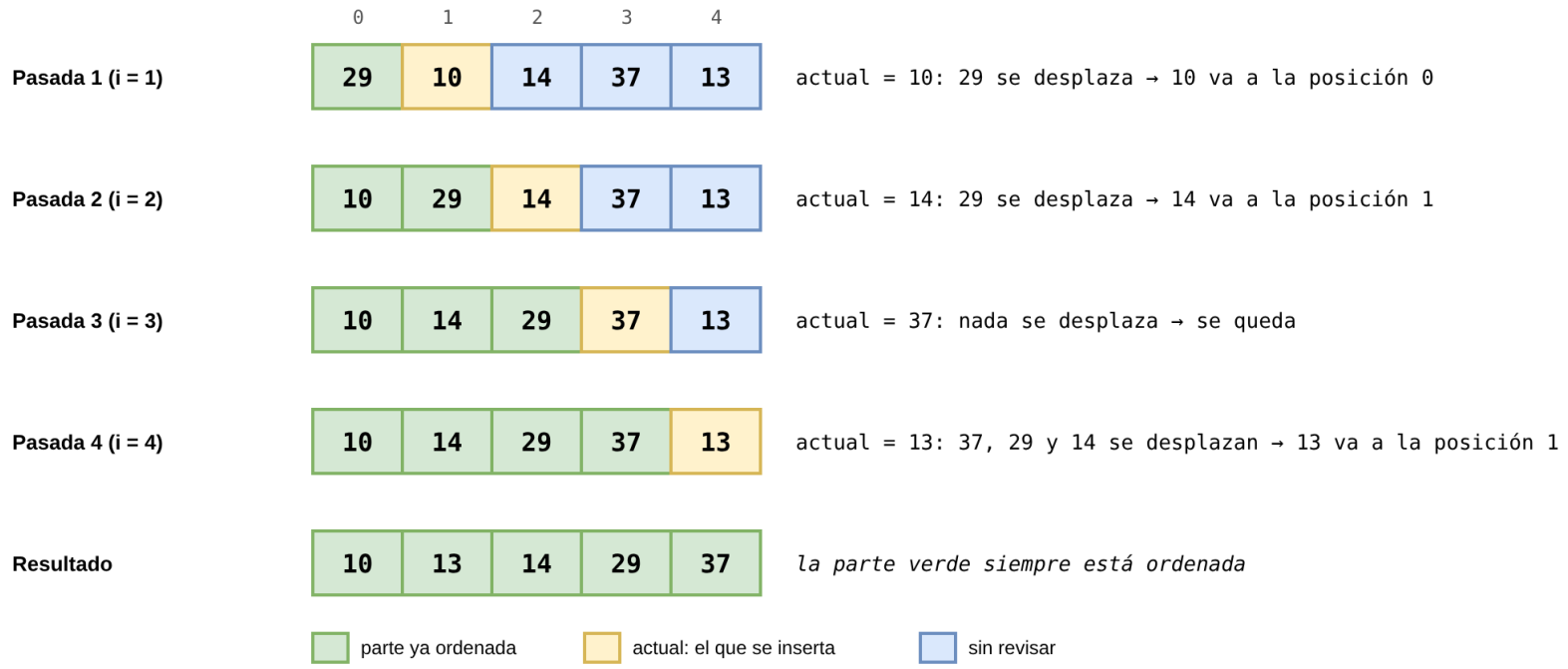

```
def ordenar_burbuja(lista):
    """
    Ordena una lista de menor a mayor con el método de la burbuja.

    Supone:
        Los elementos de lista se pueden comparar con >.
    Garantiza:
        MODIFICA lista: queda ordenada de menor a mayor. Se detiene antes
        si en una pasada no hubo intercambios. No retorna nada.
    """
    n = len(lista)
    for i in range(n - 1):
        hubo_intercambio = False
        for j in range(n - 1 - i):
            if lista[j] > lista[j + 1]:
                intercambiar(lista, j, j + 1)
                hubo_intercambio = True
        if not hubo_intercambio:
            break
```

Link simulación

Ordenamiento por inserción

- **Como al ordenar cartas en la mano**: se toma la siguiente carta y se mete en su lugar entre las que ya están ordenadas.
- **La lista se divide en dos partes**: la parte ordenada (izquierda, verde) y la parte sin revisar (derecha, azul). En cada pasada, la parte ordenada crece en uno.
- **El elemento actual (amarillo) se compara hacia la izquierda**: los mayores se desplazan una casilla y el actual queda en el hueco.



Ordenamiento por inserción: el algoritmo

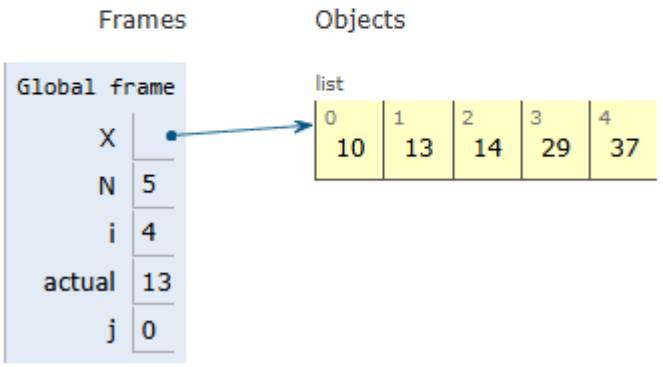
- **Ciclo externo (`i`)**: empieza en **`1`**, porque el primer elemento solo ya está "ordenado". Toma el actual **`X[i]`**.
- **Ciclo interno (`while`)**: mientras haya un elemento a la izquierda mayor que el actual, se desplaza una casilla a la derecha.
- **Al salir del `while`, el actual se coloca en el hueco (`j + 1`). No hay intercambio**: solo desplazamientos y una asignación final.

Código:

```
X = [29, 10, 14, 37, 13]
N = 5
print("inicio:", X)
for i in range(1, N):
    # guardar el actual
    actual = X[i]
    j = i - 1
    # desplazar a la derecha los mayores que el actual
    while j >= 0 and X[j] > actual:
        X[j + 1] = X[j]
        j = j - 1
    # colocar el actual en el hueco
    X[j + 1] = actual
    print("i =", i, "| actual =", actual, "| posición",
          j + 1, "|", X)
print("final:", X)
```

Print output

```
inicio: [29, 10, 14, 37, 13]
i = 1 | actual = 10 | posición 0 | [10, 29, 14, 37, 13]
i = 2 | actual = 14 | posición 1 | [10, 14, 29, 37, 13]
i = 3 | actual = 37 | posición 3 | [10, 14, 29, 37, 13]
i = 4 | actual = 13 | posición 1 | [10, 13, 14, 29, 37]
final: [10, 13, 14, 29, 37]
```



Link simulación

Inserción: el desplazamiento paso a paso

- Se toma **una sola pasada**, la última (**`i = 4`**, **`actual = 13`**), porque es la que más mueve.
- Cada vuelta del **`while`** **copia** un mayor una casilla a la derecha. Por un momento el número aparece **repetido (37, 37)**; no es un error, es el hueco que se va corriendo.
- Al final, el actual se coloca una sola vez en el hueco. **Inserción no intercambia: desplaza.**

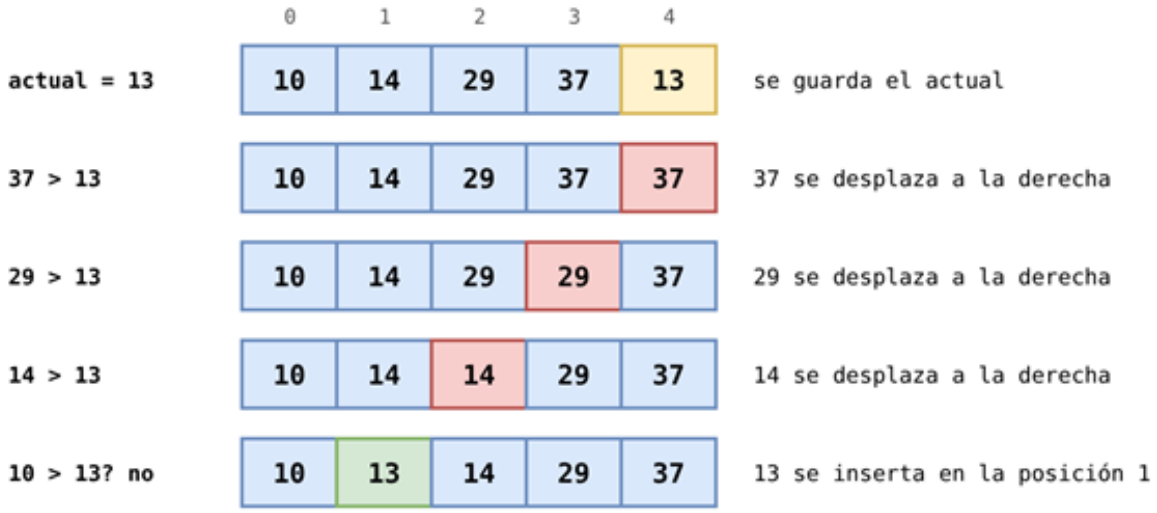
Código:

```
X = [10, 14, 29, 37, 13]
i = 4
actual = X[i]
j = i - 1
while j >= 0 and X[j] > actual:
    X[j + 1] = X[j]
    j = j - 1
    print(X)
X[j + 1] = actual
print(X)
```

Link simulación

Salida:

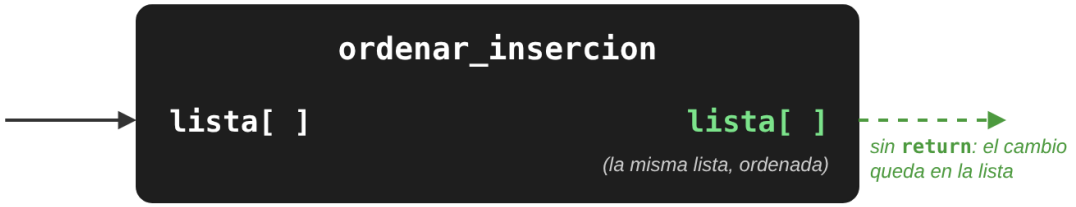
```
[10, 14, 29, 37, 37]
[10, 14, 29, 29, 37]
[10, 14, 14, 29, 37]
[10, 13, 14, 29, 37]
```



Inserción no intercambia: desplaza los mayores una casilla y al final coloca el actual.

Inserción como función

- El mismo algoritmo (el original, sin expresar como función), con **`X`** reemplazado por **`lista`** y **`N`** por **`len(lista)`**.
- **No usa `intercambiar`**: inserción desplaza, no intercambia. Es el único de los tres métodos que no la llama.
- **Caso 3**: MODIFICA la lista y no retorna nada.



ejemplo: `ordenar_insercion(X)` *deja* `X = [10, 13, 14, 29, 37]`

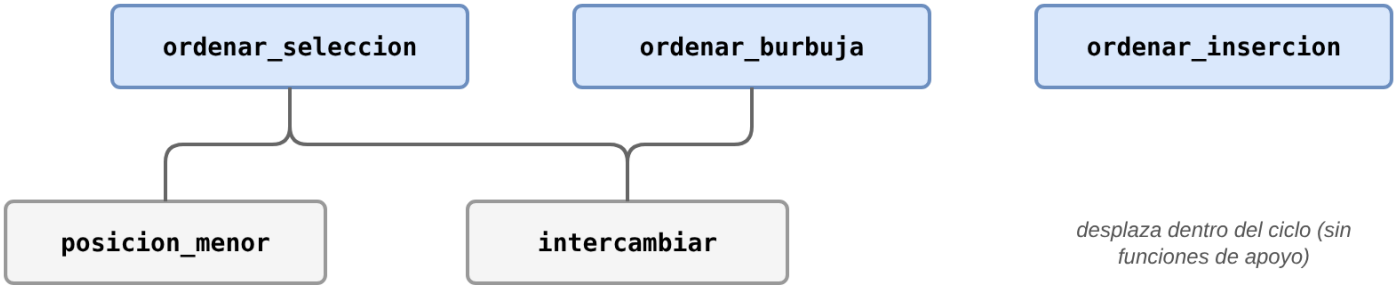
```
def ordenar_insercion(lista):
    """
    Ordena una lista de menor a mayor con el método de inserción.

    Supone:
        Los elementos de lista se pueden comparar con >.
    Garantiza:
        MODIFICA lista: queda ordenada de menor a mayor. No retorna nada.
    """
    for i in range(1, len(lista)):
        actual = lista[i]
        j = i - 1
        while j >= 0 and lista[j] > actual:
            lista[j + 1] = lista[j]
            j = j - 1
        lista[j + 1] = actual
```

Link simulación

Comparación

- **Por fuera son iguales**: los tres reciben una lista, la dejan ordenada (**MODIFICA**) y no retornan nada. Quien los llama no nota la diferencia.
- **Por dentro son distintos**:
  - **Selección** busca el menor y lo pone al inicio (**`posicion_menor`** + **`intercambiar`**).
  - **Burbuja** compara vecinos y lleva el mayor al final (**`intercambiar`**).
  - **Inserción** mete cada elemento en la parte ordenada (desplaza, sin funciones de apoyo).
- **`intercambiar`** se escribió una vez y se usa en dos métodos.



gris: funciones de apoyo (bloque 3) · azul: métodos de ordenamiento (caso 3)

```
original = [29, 10, 14, 37, 13]
A = list(original)
B = list(original)
C = list(original)
print("antes:     ", original)
ordenar_seleccion(A)
ordenar_burbuja(B)
ordenar_insercion(C)
print("selección: ", A)
print("burbuja:   ", B)
print("inserción: ", C)
print("original:  ", original)
```

```
antes:      [29, 10, 14, 37, 13]
selección:  [10, 13, 14, 29, 37]
burbuja:    [10, 13, 14, 29, 37]
inserción:  [10, 13, 14, 29, 37]
original:   [29, 10, 14, 37, 13]
```

Comparación: Cuando trabajan

- Se cuenta cuántas **comparaciones** hace cada método (cuántas veces evalúa **<** o **>** entre dos elementos).
- **Con la lista ya ordenada**, burbuja e inserción terminan rápido (4 comparaciones). Selección hace 10 igual: **siempre busca el menor completo, aunque ya esté en su lugar**.
- Con **1000 elementos al azar**, inserción hace cerca de la mitad de comparaciones que los otros dos.
- **Al revés (peor caso)**, los tres hacen lo mismo.

Método	Ya ordenado (mejor caso)	Ejemplo [29, 10, 14, 37, 13]	Al revés (peor caso)	1000 al azar
Selección	10	10	10	499 500
Burbuja	4	10	10	498 554
Inserción	4	8	10	246 325

Número de comparaciones que hace cada método. Verde: termina rápido si el arreglo ya está ordenado.

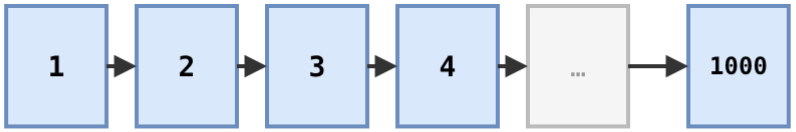
Los tres ordenan bien; la diferencia es cuánto trabajan

Búsqueda binaria: ¿Para que ordenar?

- La búsqueda lineal revisa **uno por uno**: con 1000 elementos puede necesitar 1000 comparaciones.
- Si la lista está **ordenada**, se puede hacer como con el directorio telefónico: abrir en la mitad, ver si el buscado está antes o después y descartar la otra mitad.
- Cada comparación **descarta la mitad** de lo que queda. **Requisito**: la lista debe estar ordenada (de menor a mayor).

Guía telefónica de **1000** páginas, ordenada por apellido. Se busca a **"Pérez"**.

1. Página por página

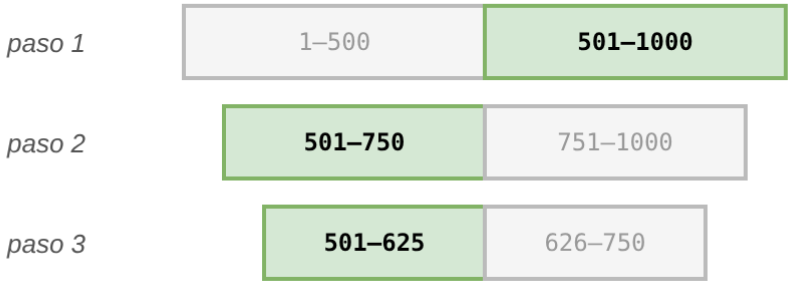


se revisa una página tras otra

- hasta **1000** pasos
- sirve aunque la guía esté **en desorden**

búsqueda lineal

2. Abrir por la mitad



`1000 → 500 → 250 → 125 → … → 1 página`

verde: la mitad que se conserva · gris: la que se descarta

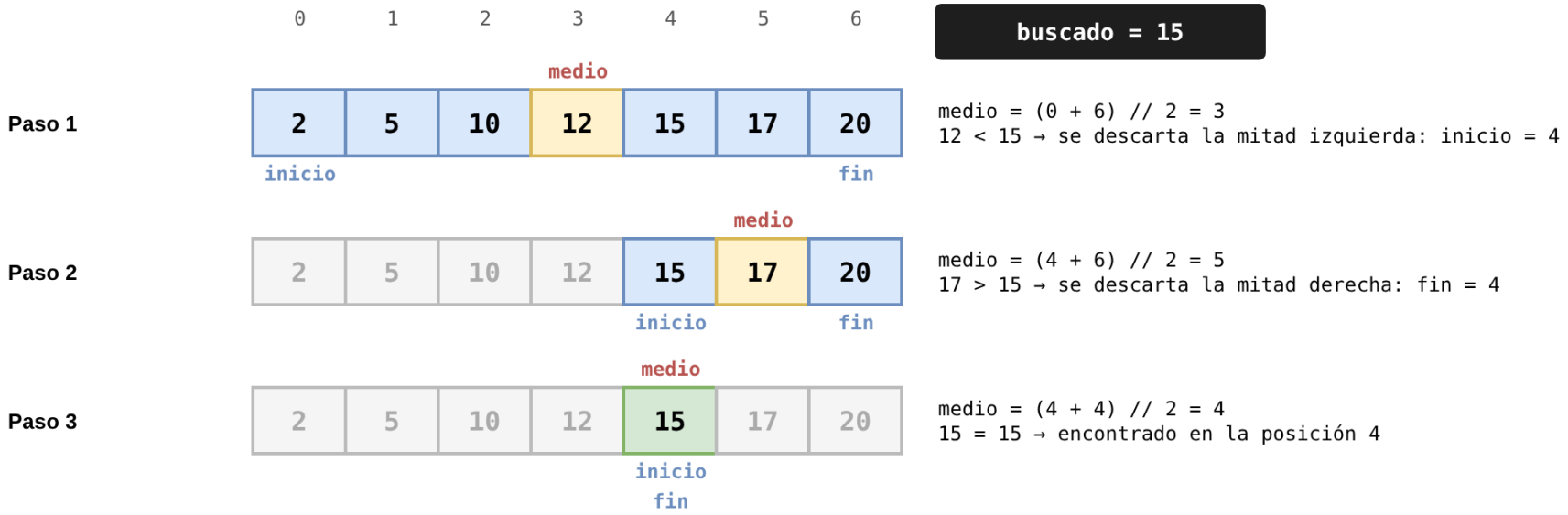
- unos **10** pasos
- solo si está **ordenada**

búsqueda binaria

Idea tomada de CS50x, Lecture 3 (link)

Búsqueda binaria: Paso a paso.

- **Tres variables marcan la parte que todavía se revisa**: **`inicio`**, **`fin`** y **`medio = (inicio + fin) // 2`**.
- Se compara el buscado con **`X[medio]`**:
  - **Igual**: encontrado.
  - **`X[medio]` menor**: el buscado está a la derecha: `inicio = medio + 1`.
  - **`X[medio]` mayor**: está a la izquierda: `fin = medio - 1`.
- Si **`inicio`** pasa a **`fin`**, no queda nada por revisar: **no está**.



3 pasos en lugar de 5: en cada paso se descarta la mitad de lo que queda (gris).

Búsqueda binaria: el algoritmo

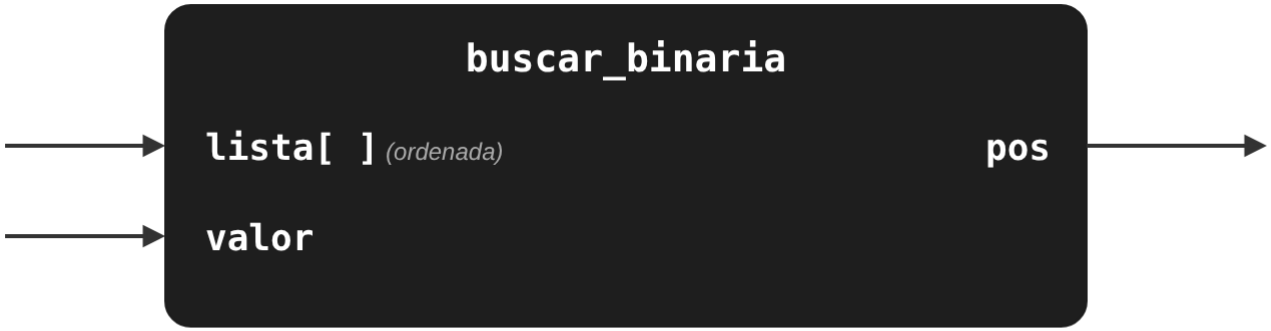
- **`while inicio <= fin`**: se repite mientras quede algo por revisar. Es un **`while`** y no un **`for`**, porque no se sabe de antemano cuántos pasos habrá.
- **`if / elif / else`**: son los tres casos (encontrado, descartar izquierda, descartar derecha).
- **`pos = -1` al inicio y `break` al encontrarlo**: el mismo patrón de la búsqueda lineal.

```
X = [2, 5, 10, 12, 15, 17, 20]
N = 7
buscado = 15 # Probar con 13 (Caso no encontrado)
inicio = 0
fin = N - 1
pos = -1
while inicio <= fin:
    medio = (inicio + fin) // 2
    print("inicio =", inicio, "| fin =", fin, "| medio =", medio, "| X[medio] =", X[medio])
    if X[medio] == buscado:
        # encontrado
        pos = medio
        break
    elif X[medio] < buscado:
        # descartar la mitad izquierda
        inicio = medio + 1
    else:
        # descartar la mitad derecha
        fin = medio - 1
print("al salir: inicio =", inicio, "| fin =", fin, "| pos =", pos)
```

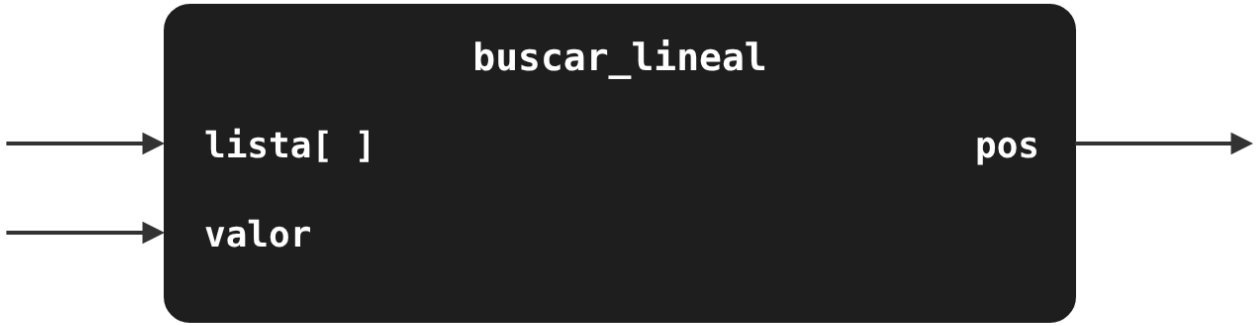
Link simulación

Búsqueda binaria como función

- **Caso 1**: no modifica la lista y retorna un valor, la posición o -1.
- **`fin = len(lista) - 1`** reemplaza a **`N - 1`**.
- **El Supone es nuevo y obligatorio**: la lista debe estar ordenada de menor a mayor. La función no lo revisa; es responsabilidad de quien la llama.

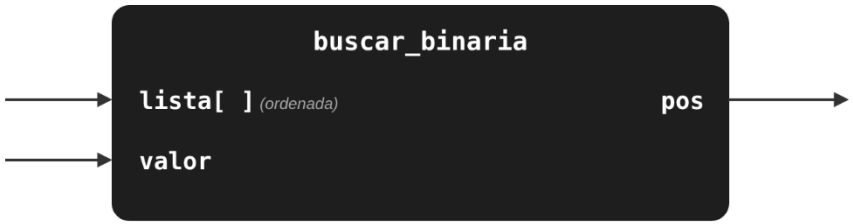


ejemplo: `buscar_binaria([2, 5, 10, 12, 15, 17, 20], 15)` *devuelve* **4**



ejemplo: `buscar_lineal([5.0, 4.0, 3.5, 2.87], 3.5)` *devuelve* **2**

Búsqueda binaria como función



ejemplo: `buscar_binaria([2, 5, 10, 12, 15, 17, 20], 15)` *devuelve* **4**

```
def buscar_binaria(lista, valor):
    """
    Busca un valor en una lista ordenada, partiéndola a la mitad en cada paso.

    Supone:
        lista está ordenada de menor a mayor.
    Garantiza:
        Retorna una posición donde está valor, o -1 si valor no está.
        No modifica lista.
    """
    inicio = 0
    fin = len(lista) - 1
    pos = -1
    while inicio <= fin:
        medio = (inicio + fin) // 2
        if lista[medio] == valor:
            pos = medio
            break
        elif lista[medio] < valor:
            inicio = medio + 1
        else:
            fin = medio - 1
    return pos
```

Link simulación

Pruebas de `buscar_binaria`

- **Se planean igual que en `buscar_lineal`**: casos típicos, límite y especiales. Todos con la lista ordenada, porque así lo exige el Supone.

Tabla (con **`X = [2.87, 3.5, 4.0, 5.0]`**):

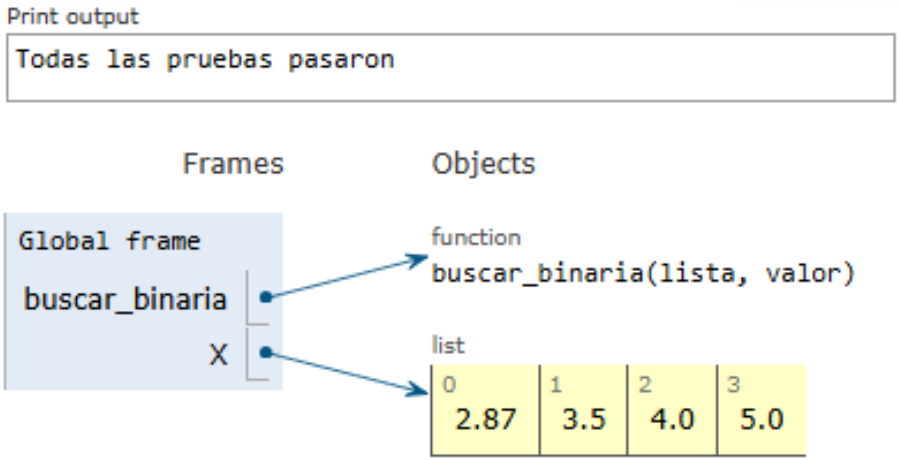
Caso	Tipo	Resultado esperado
3.5 en X	**Típico**: está en el medio	1
2.87 en X	**Límite**: es el primero	0
5.0 en X	**Límite**: es el último	3
1.0 en X	**Especial**: menor que todos	-1
7.0 en X	**Especial**: mayor que todos	-1
3.7 en X	**Especial**: en un hueco	-1
2 en [1, 2, 2, 2, 3]	**Especial**: está repetido	2 (una de ellas, no necesariamente la primera)
3.5 en []	**Especial**: lista vacía	-1

Pruebas de `buscar_binaria`

- **Se planean igual que en `buscar_lineal`**: casos típicos, límite y especiales. Todos con la lista ordenada, porque así lo exige el Supone.

```
X = [2.87, 3.5, 4.0, 5.0]
assert buscar_binaria(X, 3.5) == 1          # típico: está en el medio
assert buscar_binaria(X, 2.87) == 0         # límite: el primero
assert buscar_binaria(X, 5.0) == 3          # límite: el último
assert buscar_binaria(X, 1.0) == -1         # especial: menor que todos
assert buscar_binaria(X, 7.0) == -1         # especial: mayor que todos
assert buscar_binaria(X, 3.7) == -1         # especial: en un hueco
assert buscar_binaria([1, 2, 2, 2, 3], 2) == 2   # especial: repetido -> una de ellas
assert buscar_binaria([], 3.5) == -1        # especial: lista vacía
# Así NO (descomentar en clase): lista de mayor a menor, no cumple el Supone
# assert buscar_binaria([5.0, 4.0, 3.5, 2.87], 3.5) == 2
print("Todas las pruebas pasaron")
```

Link simulación



Búsqueda lineal frente a binaria

- En el peor caso, la búsqueda **lineal** revisa **todos** los elementos; la **binaria descarta la mitad** en cada paso.
- Si se duplica **N**, la lineal hace el doble de comparaciones y la binaria solo una más.
- **¿Cuál usar?**
  - **Lineal**: si la lista no está ordenada o se va a buscar una sola vez.
  - **Binaria**: si la lista ya está ordenada o se va a buscar muchas veces (se ordena una vez y se busca rápido siempre).

Tamaño N	Búsqueda lineal (peor caso)	Búsqueda binaria (peor caso)
10	10	4
100	100	7
1 000	1 000	10
1 000 000	1 000 000	20
1 000 000 000	1 000 000 000	30

Número máximo de comparaciones. La binaria necesita el arreglo ordenado.

Integrando lo visto: administración de notas de un curso

- Un curso admite máximo **`N = 10`** estudiantes. De cada uno se guarda su **id (texto)**, su **nombre** y su **nota**.
- Por ahora hay 4 inscritos; se pueden agregar más hasta llenar el cupo.
- Se necesita un programa para administrar el curso: agregar, eliminar, actualizar, buscar, listar y **ordenar**.

Id	Nombre	Nota
"000"	"Ana"	3.0
"123"	"Beto"	4.5
"987"	"Carla"	2.0
"007"	"Dario"	4.0
	(libre)	
	...	
	(libre)	

4 inscritos

6 cupos libres (hasta N = 10)

Administración de notas de un curso: El menú

- El programa muestra un menú y repite hasta que se elige **0. Salir**.
- Cada opción es una función que recibe los vectores y cantidad, y usa lo visto en clases anteriores.
- En esta clase se implementan en detalle las opciones **6** y **7** (**ordenar**). El programa completo está en el repositorio.

Salida tentativa en consola

```
=== Administración del curso ===
1. Agregar estudiante
2. Eliminar estudiante
3. Actualizar nota
4. Buscar estudiante
5. Listar curso
6. Ordenar por nota
7. Ordenar por nombre
0. Salir

Opción:
```

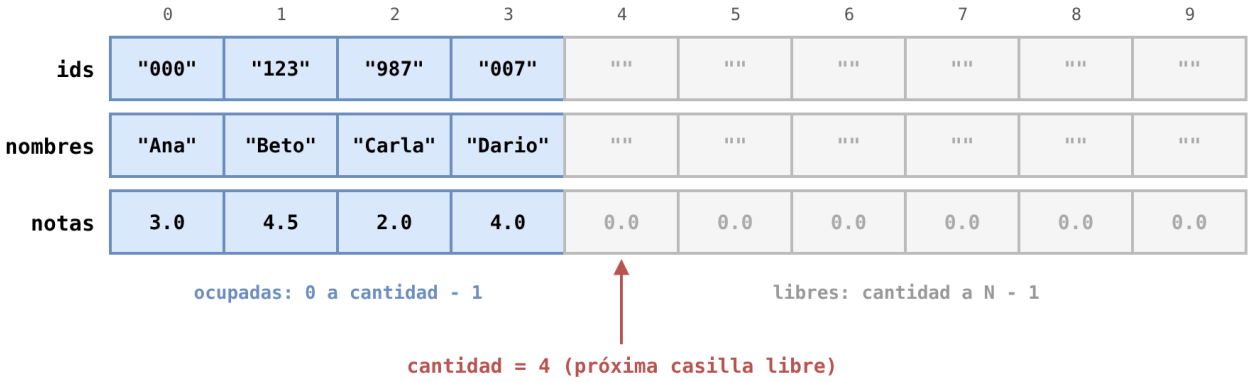
Administración de notas de un curso: El menú

- Cada opción del menú se resuelve con **lo visto en el curso**.
- Las opciones **6 y 7 (ordenar)** son el tema de esta clase y se implementan en detalle en las diapositivas siguientes.

Opción	**Qué hace**	**Usa**
1. Agregar	Agrega al final si hay cupo y el Id no existe	cantidad, búsqueda lineal
2. Eliminar	Quita un estudiante por Id y corre los siguientes a la izquierda	búsqueda lineal, desplazamiento
3. Actualizar nota	Cambia la nota de un Id	búsqueda lineal
4. Buscar	Muestra los datos de un Id	búsqueda lineal
5. Listar	Muestra los **`cantidad`** inscritos	recorrido
6. Ordenar por nota	**De mayor a menor**	**selección + intercambio**
7. Ordenar por nombre	**Alfabético**	**selección + intercambio**

El arreglo parcialmente lleno

- Se crean tres vectores de tamaño **N** (el cupo), aunque al principio no haya nadie.
- cantidad dice cuántas casillas están ocupadas: de **0** a **cantidad - 1**. También es la próxima casilla libre.
- Por eso las funciones reciben cantidad y recorren hasta ahí, no hasta **len()**. Si no, tratarían las casillas vacías como estudiantes.



Cada vector tiene N = 10 casillas; solo las primeras 4 (cantidad) tienen datos.

```
N = 10
ids = [""] * N
nombres = [""] * N
notas = [0.0] * N
cantidad = 0
cantidad = agregar_estudiante(ids, nombres, notas, cantidad, "000", "Ana", 3.0)
cantidad = agregar_estudiante(ids, nombres, notas, cantidad, "123", "Beto", 4.5)
cantidad = agregar_estudiante(ids, nombres, notas, cantidad, "987", "Carla", 2.0)
cantidad = agregar_estudiante(ids, nombres, notas, cantidad, "007", "Dario", 4.0)
print(cantidad)
print(nombres)
```

Link simulación

Primer intento: `ordenar_seleccion(notas)`

- Se usa la función **`ordenar_seleccion`** (vista previamente), tal como está.
- **Resultado**: todos quedaron con **`0.0`**.
- **¿Por qué?** ordenar_seleccion recorre hasta **`len(notas)`**, es decir, las 10 casillas. Los seis **`0.0`** de las casillas libres son los menores y se **van al principio**.

```
mostrar_listado(ids, nombres, notas, cantidad)
ordenar_seleccion(notas)      # primer intento: ordenar solo notas
print("después:")
mostrar_listado(ids, nombres, notas, cantidad)
```

Link simulación

Antes

Id	Nombre	Nota
"000"	"Ana"	3.0
"123"	"Beto"	4.5
"987"	"Carla"	2.0
"007"	"Dario"	4.0

Después ✗

Id	Nombre	Nota
"000"	"Ana"	0.0
"123"	"Beto"	0.0
"987"	"Carla"	0.0
"007"	"Dario"	0.0

Segundo intento: hasta `cantidad`, pero solo `notas`

- **Se corrige el primer error**: se busca la mayor nota entre **`i`** y **`cantidad - 1`**, con **`posicion_mayor_nota`**. Así se queda dentro de las casillas ocupadas y se ordena de mayor a menor.
- **Las notas quedan bien ordenadas**: 4.5, 4.0, 3.0, 2.0.
- **Pero los nombres no se movieron**: ahora Ana "tiene" el 4.5 de Beto y Dario "tiene" el 2.0 de Carla.

```
def posicion_mayor_nota(notas, inicio, cantidad):
    pos_mayor = inicio
    for j in range(inicio + 1, cantidad):
        if notas[j] > notas[pos_mayor]:
            pos_mayor = j
    return pos_mayor

for i in range(cantidad - 1):
    pos_mayor = posicion_mayor_nota(notas, i, cantidad)
    intercambiar(notas, i, pos_mayor)      # solo notas
```

Link simulación

Lo correcto ✓

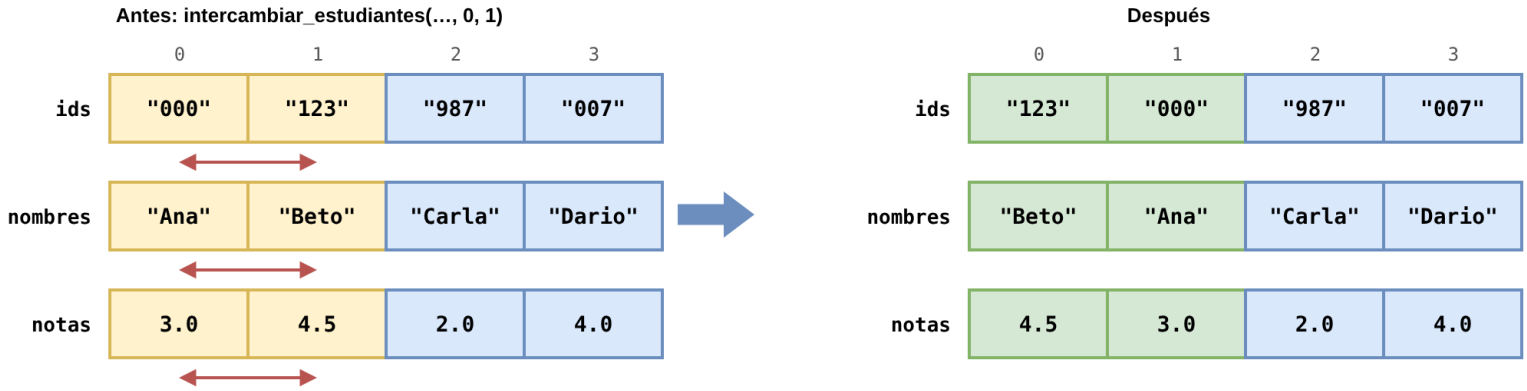
Id	Nombre	Nota
"123"	"Beto"	4.5
"007"	"Dario"	4.0
"000"	"Ana"	3.0
"987"	"Carla"	2.0

Lo que imprime ✗

Id	Nombre	Nota
"000"	"Ana"	4.5
"123"	"Beto"	4.0
"987"	"Carla"	3.0
"007"	"Dario"	2.0

La solución: intercambiar en los tres vectores

- Un estudiante **ocupa la misma posición en los tres vectores**. Para moverlo, hay que moverlo en los tres.
- **`intercambiar_estudiantes`** llama tres veces a **`intercambiar`**, una por vector y con las mismas posiciones.
- **Caso 3**: MODIFICA los tres vectores y no retorna nada.



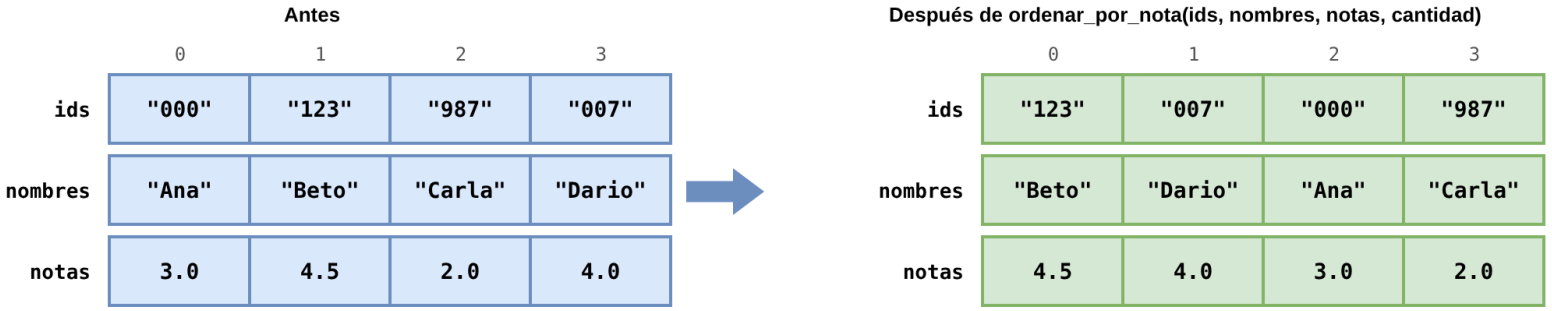
El estudiante de la posición 1 pasa a la 0 (y viceversa) en los TRES vectores: cada Id y nombre sigue unido a su nota.

```
def intercambiar_estudiantes(ids, nombres, notas, i, j):
    """
    Supone: i y j son posiciones válidas de los tres arreglos.
    Garantiza: MODIFICA ids, nombres y notas: los estudiantes de las
    posiciones i y j quedan intercambiados en los tres. No retorna nada.
    """
    intercambiar(ids, i, j)
    intercambiar(nombres, i, j)
    intercambiar(notas, i, j)
```

Link simulación

ordenar_por_nota: la selección aplicada al curso

- Es la función selección vista previamente, con tres cambios:
  - **posicion_mayor_nota en vez de posicion_menor**: de mayor a menor.
  - **cantidad en vez de len()**: solo las casillas ocupadas.
  - **intercambiar_estudiantes en vez de intercambiar**: el estudiante completo.
- **Caso 3**: MODIFICA los tres vectores y no retorna nada.



De mayor a menor nota, y cada Id y nombre sigue en la misma posición que su nota.

```
def ordenar_por_nota(ids, nombres, notas, cantidad):
    """
    Ordena el curso de mayor a menor nota (selección).
    Supone:
        0 <= cantidad <= len(ids).
    Garantiza:
        MODIFICA ids, nombres y notas: los inscritos quedan de mayor a
        menor nota, y cada estudiante conserva su Id, nombre y nota.
        Las posiciones libres no cambian. No retorna nada.
    """
    for i in range(cantidad - 1):
        pos_mayor = posicion_mayor_nota(notas, i, cantidad)
        intercambiar_estudiantes(ids, nombres, notas, i, pos_mayor)
```

Link simulación

ordenar_por_nombre: mismo método, otro criterio

- **Es la misma función que `ordenar_por_nota`**. Solo cambia qué se compara: **`nombres`** con **`<`** (orden alfabético) en lugar de **`notas`** con **`>`**.
- **Python compara textos letra por letra**, como en el diccionario: **`"Ana" < "Beto"`** es **`True`**.
- **Para más adelante**: las dos funciones son casi idénticas. Existe una forma de escribir una sola y pasarle la comparación como parámetro. Se verá en otro curso.

```
def posicion_menor_nombre(nombres, inicio, cantidad):
    pos_menor = inicio
    for j in range(inicio + 1, cantidad):
        if nombres[j] < nombres[pos_menor]:
            pos_menor = j
    return pos_menor

def ordenar_por_nombre(ids, nombres, notas, cantidad):
    for i in range(cantidad - 1):
        pos_menor = posicion_menor_nombre(nombres, i, cantidad)
        intercambiar_estudiantes(ids, nombres, notas, i, pos_menor)
```

Link simulación

Antes (por nota)

Id	Nombre	Nota
"123"	"Beto"	4.5
"007"	"Dario"	4.0
"000"	"Ana"	3.0
"987"	"Carla"	2.0

Después (por nombre)

Id	Nombre	Nota
"000"	"Ana"	3.0
"123"	"Beto"	4.5
"987"	"Carla"	2.0
"007"	"Dario"	4.0

Pruebas de `ordenar_por_nota` y `ordenar_por_nombre`

- **Se planean igual que siempre**: casos típicos, límite y especiales.
- **Hay algo nuevo que probar**: cada estudiante conserva su Id, nombre y nota, y las casillas **libres no se tocan**.

Caso	**Tipo**	**Resultado esperado**
Curso de ejemplo, por nota	Típico	notas: 4.5, 4.0, 3.0, 2.0; nombres e Id se mueven con su nota
El mismo curso, por nombre	Típico	Ana, Beto, Carla, Dario, cada uno con su nota
Las 2 casillas libres	Especial	Siguen en "" y 0.0
Notas repetidas (Ana 4.0, Beto 3.0, Carla 4.0)	Especial	notas: 4.0, 4.0, 3.0; Ana y Carla arriba, cada una con su 4.0
Un solo estudiante	Límite	No cambia nada
Curso vacío (cantidad = 0)	Límite	No cambia nada

Pruebas de `ordenar_por_nota` y `ordenar_por_nombre`

```
# Típico: por nota (las casillas libres no se tocan)
ids = ["000", "123", "987", "007", "", ""]
nombres = ["Ana", "Beto", "Carla", "Dario", "", ""]
notas = [3.0, 4.5, 2.0, 4.0, 0.0, 0.0]
ordenar_por_nota(ids, nombres, notas, 4)
assert notas == [4.5, 4.0, 3.0, 2.0, 0.0, 0.0]
assert nombres == ["Beto", "Dario", "Ana", "Carla", "", ""]
assert ids == ["123", "007", "000", "987", "", ""]

# Típico: por nombre (se parte del orden por nota)
ordenar_por_nombre(ids, nombres, notas, 4)
assert nombres == ["Ana", "Beto", "Carla", "Dario", "", ""]
assert notas == [3.0, 4.5, 2.0, 4.0, 0.0, 0.0]

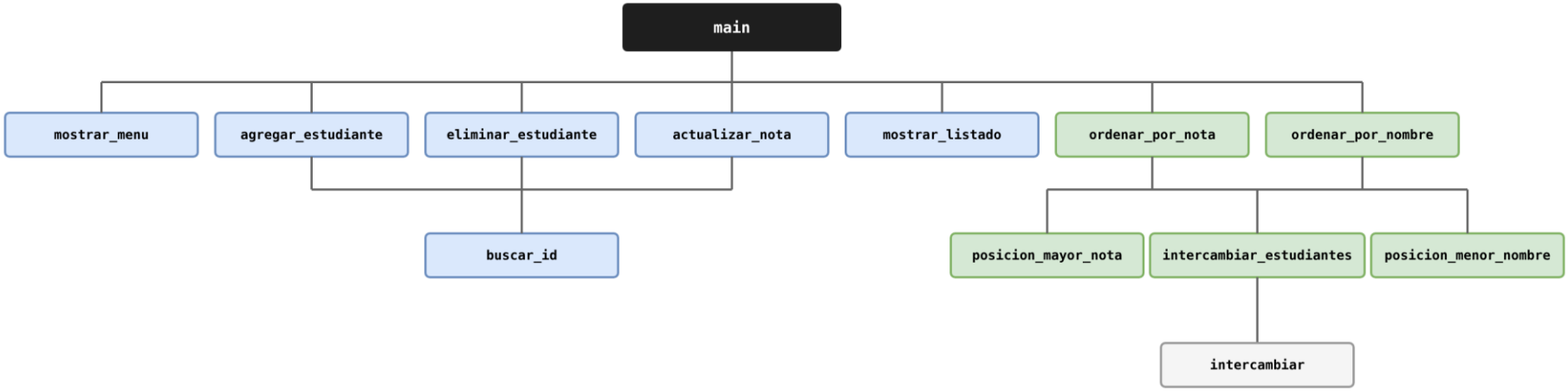
# Especial: notas repetidas (cada nombre sigue con su nota)
ids = ["1", "2", "3"]
nombres = ["Ana", "Beto", "Carla"]
notas = [4.0, 3.0, 4.0]
ordenar_por_nota(ids, nombres, notas, 3)
assert notas == [4.0, 4.0, 3.0]
assert nombres == ["Ana", "Carla", "Beto"]

# Límite: un solo estudiante y curso vacío
nombres = ["Ana", ""]
notas = [3.0, 0.0]
ordenar_por_nota(["1", ""], nombres, notas, 1)
assert nombres == ["Ana", ""]
ordenar_por_nota(["", ""], ["", ""], notas, 0)
assert notas == [3.0, 0.0]

print("Todas las pruebas pasaron")
```

Administración de notas: el diseño

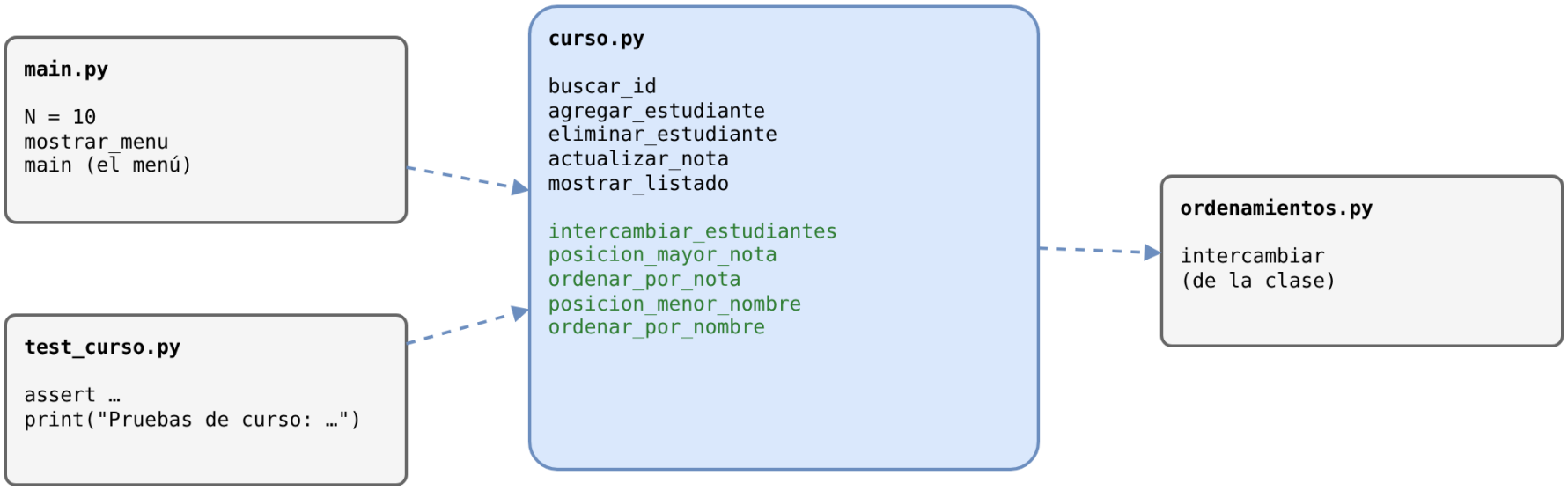
- **`main`** muestra el menú y llama una **función por opción** (**diseño top-down**).
- **En verde, la parte de ordenar**, que es la que se explicó en esta clase. El resto (en azul) está en el repositorio.
- **función `intercambiar`, sigue abajo de todo**: se reutiliza también aquí.



verde: la parte de ordenar (esta clase) · azul: resto del menú (repositorio) · gris: reutilizada de ordenamientos.py

Administración de notas: archivos y ejecución.

- Los mismos tres archivos de siempre: **`curso.py`** (las funciones), **`test_curso.py`** (las pruebas) y **`main.py`** (el menú).
- **`curso.py`** importa intercambiar de **`ordenamientos.py`**.



Flechas punteadas: import. En verde, las funciones de ordenar que se explican en esta clase.

- **Ejecución en vivo**: **`python test_curso.py`** y después **`python main.py`**.

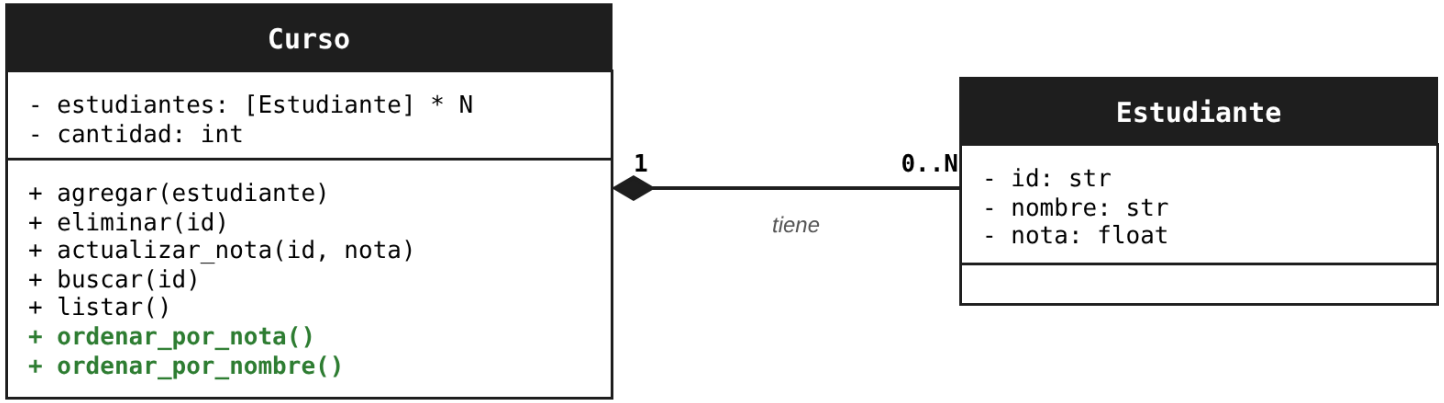
¿Qué fue incómodo?

- **Un estudiante no existe como tal**. Son tres vectores que comparten una posición, y si se mueve uno sin los otros, los datos se mezclan sin ningún aviso.
- **Cada intercambio son tres**. **`intercambiar_estudiantes`** tiene que acordarse de los tres vectores.
- **`cantidad` va a todas partes**. varias funciones la reciben, y las que la cambian tienen que modificar los vectores y además retornarla.
- **Una función de ordenar por cada criterio**, casi idénticas.

¿Qué pasa si mañana se agrega el correo de cada estudiante?
Respuesta: un vector más, varias funciones de **`curso.py`** cambian, además del **`main`**: agregar, eliminar, listar, intercambiar y los dos ordenamientos.

La respuesta (próxima clase): un estudiante es una sola cosa

- Con **POO (Programación Orientada a Objetos)** se define un **`Estudiante`** que guarda juntos su **Id**, su **nombre** y su **nota**, y un **`Curso`** que tiene a sus estudiantes.
- **El menú es el mismo**: las mismas siete opciones, ahora como métodos de **`Curso`**.



Un estudiante es UNA sola cosa: intercambiar dos estudiantes es un solo intercambio, sin importar cuántos datos tenga.

Incomodidad	**Con objetos**
Un estudiante no existe	Existe: es un Estudiante
Cada intercambio son tres	Se intercambia un Estudiante completo
cantidad va a todas partes	Vive dentro de **`Curso`**
¿Y el correo?	Una línea más en **`Estudiante`**
Una función por criterio	**Sigue igual** (se resuelve de otra forma, más adelante)

Otra pregunta: ¿y si hay varias notas por estudiante?

- Hasta ahora cada estudiante tiene una nota. En un curso real hay **varios parciales**.
- ¿Un vector por parcial (**`notas1`**, **`notas2`**, **`notas3`**…)? Serían más vectores paralelos, y con ellos más incomodidad.
- **Mejor**: una **tabla de notas**, donde la **fila `i`** es un estudiante y la **columna `j`** es una evaluación. Eso es una matriz: **`notas[i][j]`**.

		Parcial 1 `j = 0`	Parcial 2 `j = 1`	Parcial 3 `j = 2`
Ana	`i = 0`	**3.0**	**3.5**	**2.5**
Beto	`i = 1`	**4.5**	**4.0**	**5.0**
Carla	`i = 2`	**2.0**	**3.0**	**3.5**
Dario	`i = 3`	**4.0**	**4.5**	**3.0**

`notas[2][1] = 3.0`

fila i: un estudiante · columna j: una evaluación · todas las casillas son notas (mismo tipo)

- **POO**: datos distintos de una misma cosa
- **Matrices**: muchos datos del mismo tipo en dos dimensiones

Lo que Python ya trae: `sorted()` y `.sort()`

- Python ya tiene ordenamiento incorporado. **Ahora se sabe qué hace por dentro**.
- **`sorted(lista)`** retorna una **lista nueva** ordenada; la original no cambia. Equivale a copiar y ordenar la copia.
- **`lista.sort()`** **modifica** la lista y no retorna nada: es el caso 3, como **`ordenar_seleccion`**.
- Con **`reverse=True`** ordena de mayor a menor.

Lo que se hizo en clase	Lo que Python ya trae
`copia = list(notas)` y **`ordenar_seleccion(copia)`**	**`copia = sorted(notas)`**
`ordenar_seleccion(notas)` (MODIFICA)	**`notas.sort()`** (MODIFICA)
`posicion_mayor_nota`, de mayor a menor	**`reverse=True`**

```
notas = [3.0, 4.5, 2.0, 4.0]
ordenada = sorted(notas)     # retorna una lista nueva
print(ordenada)              # [2.0, 3.0, 4.0, 4.5]
print(notas)                 # [3.0, 4.5, 2.0, 4.0]  (no cambia)
notas.sort()                 # modifica la lista, no retorna nada
print(notas)                 # [2.0, 3.0, 4.0, 4.5]
print(sorted(notas, reverse=True))   # [4.5, 4.0, 3.0, 2.0]
```

Link simulación

Resumen

- **Búsqueda lineal**: revisa uno por uno; sirve con cualquier lista.
- **Intercambio**: tres líneas con **`temp`**, convertidas en la función **`intercambiar`**, que se usó en toda la clase.
- **Selección, burbuja e inserción**: por fuera son la misma caja negra (caso 3, MODIFICA); por dentro trabajan distinto. Inserción es la que menos trabaja con listas casi ordenadas.
- **Búsqueda binaria**: descarta la mitad en cada paso, pero exige la lista ordenada (el Supone como contrato).
- **Arreglos paralelos**: un estudiante ocupa la misma posición en varios vectores, que hay que mover juntos y recorrer hasta **`cantidad`**.

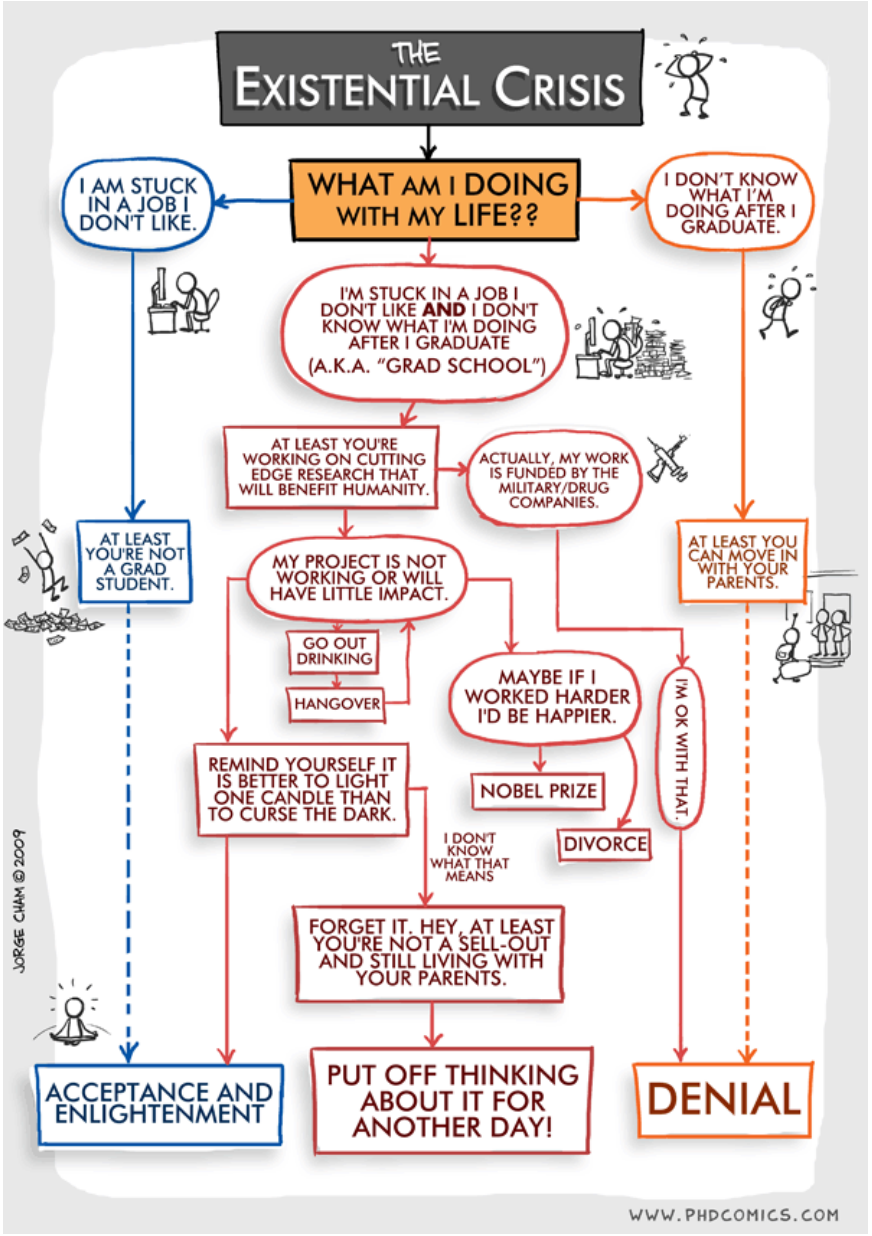
Lo que viene: POO (un estudiante es una sola cosa) y matrices (muchos datos del mismo tipo en dos dimensiones).

Referencias

- **Mera Banguero, C. A.** Material del curso Lógica y Representación I, Universidad de Antioquia. Métodos de ordenamiento y búsqueda.
- **Python Software Foundation**. Sorting Techniques (documentación oficial de Python). Para sorted(), .sort() y reverse=True. https://docs.python.org/3/howto/sorting.html
- **Material de Algoritmos y Programación, Politécnico Colombiano JIC.** Sesiones presenciales, clases 15 y 16. https://github.com/algoritmos-poli/sesiones_presenciales
- **CS50x 2026, Lecture 3: Algorithms.** Harvard University. Búsqueda lineal y binaria, selección, burbuja y el directorio telefónico. https://cs50.harvard.edu/x/2026/notes/3/
- Lambert, K. A. Fundamentals of Python: Data Structures, 2.ª ed., cap. 3 ("Searching, Sorting, and Complexity Analysis"). Cengage. La función swap como pieza reutilizable de los ordenamientos.
- Chand John, *How an Array of Integers Gets Stored in Memory*, CS Comics, 2018. https://www.cs-comics.com/how-an-array-of-integers-gets-stored-in-memory/
- Python Tutor. https://pythontutor.com

Eso es todo amigos





https://phdcomics.com/comics/archive.php?comicid=1431